实验报告-探究数据库系统与传统文件读写间的表现差异

引言

研究正文

- 实验数据与环境
  - 实验环境
    - 实验设备配置
    - 实验对象属性
  - 实验数据
- 实验过程
  - 基本数据操作对比
    - 数据插入操作
    - 执行filmdb.sql文件以创建并插入数据
    - PostgreSQL系统中不同数据插入方式测试

1. 使用Awful Loader进行数据导入操作
2. 使用Very Bad Loader进行数据导入操作
3. 使用Bad Loader进行数据导入操作
4. 使用Average Loader进行数据导入操作
5. 使用Good Loader进行数据导入操作
6. 采用多行插入语句批量进行数据导入操作
7. 使用copy关键字进行数据导入操作
8. 采用无日志表进行加速
9. 关闭索引与约束

    - 数据查询操作
    - 查询movies表中title字段包含"XXX"的元组
    - 对movies表中主键数据进行查询操作
    - 对movies表中不同数据进行频繁查询
    - 大数据量表查询测试

数据分页

缓存与缓存替换策略
索引结构
数据更新操作
数据一致性探究
事务模型
手动编程文件操作（无事务模型）方式下系统故障对数据的影响实验
数据库系统开启事务模型方式下系统故障对数据的影响实验
外键模型
外键约束与冲突处理实验
导入操作外键约束实验
总结
数据完整性探究
单表完整性检查
非法插入测试
非法修改测试
非法删除测试
总结
表损坏检查与修复
物理备份
逻辑备份
1. 基本逻辑备份操作
2. 不同数据量下逻辑备份与恢复的速度测试
3. 逻辑备份操作对数据库运行影响测试
PITR备份
PITR备份有效性实验
块数据校验和
块数据校验和有效性实验
块数据校验和性能影响实验
总结
数据安全性探究
用户权限管理有效性实验
总结

参考文献

姓名：黄政东

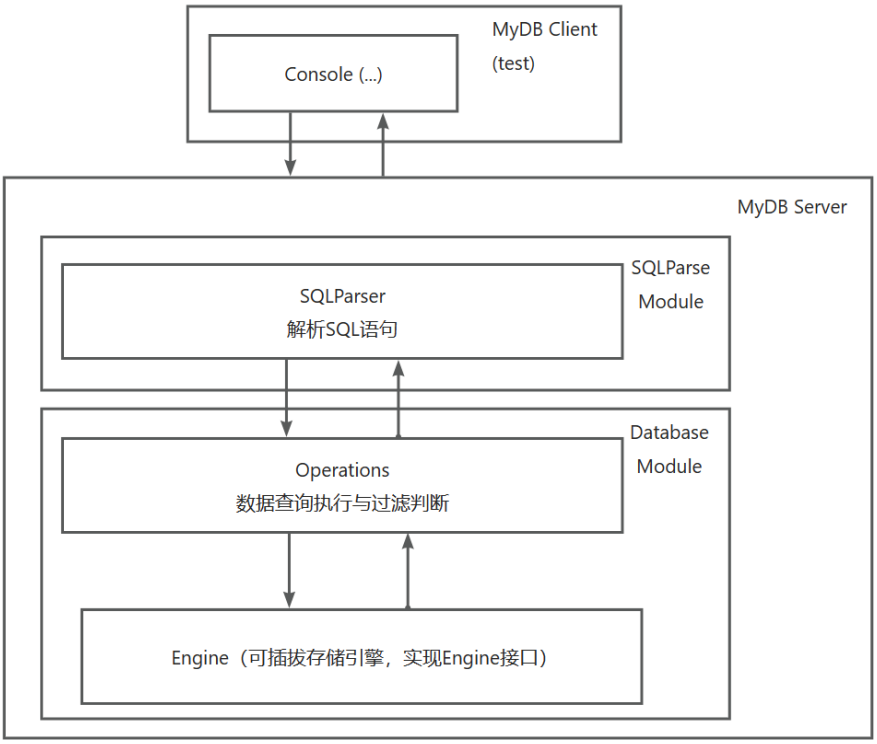
学号：12212230

引言

数据库技术是从上世纪60年代兴起的一种数据管理技术，是数据管理走向专业化的重要标志。数据库技术采用数据库对数据进行管理，相较于人工管理或手动编程的方式对数据进行处理，通过将数据管理操作抽象为可用数据库系统完成的一系列指令，采用数据库系统进行数据管理有效地实现了数据管理操作实现了规范化发展。数据库系统作为数据管理操作的专业化软件，其最为重要与核心的功能便是实现一系列的**基础数据操作**，如数据的插入、更新与查询。与此同时，数据库系统需实现**对内部数据的持久化存储**与**一致性保障**，使数据能够避免因断电、系统故障等问题而发生丢失或损坏。除此之外，数据库系统需实现对**内部数据完整性的保障**，从而保证数据操作的正常进行。最后，数据库系统还需要实现对**数据的安全存储**，在一定程度上避免数据丢失与泄露等风险。

本研究将从**基础数据操作、数据一致性、数据完整性、数据安全性**四个方面出发，**探究数据库系统与传统手动编程文件读写实现数据操作过程中的表现差异，分析数据库系统在数据管理上的优势与其原因，从而为如何更好的实现数据库数据管理提出建议**。

在实验过程中，实验将采用Java手动编程的方式对CSV文件进行读写操作，与现有常用的关系型数据库系统PostgreSQL进行对比。同时，考虑到手动编程的方式可能需要对每条查询操作进行重新编程，代码复用性不佳且局限了可测试的查询功能。本研究同时开发了基于Java的关系型数据库模拟系统（MyDB），该系统参考了数据库系统设计实现了常见的数据查询接口，同时支持对存储引擎的可插拔更换。同时系统并参考火山模型运用Antlr语法解析器实现对sql语句的语法分析，初步实现了基本数据操作与含复杂条件的SQL语句的查询操作，从而便于与现有数据库系统开展对比试验。MyDB系统的设计架构如下



研究正文

实验数据与环境

实验环境

实验设备配置

处理器：12th Gen Intel(R) Core(TM) i7–12700H

内存（RAM）：16.0 GB

存储硬盘：NVMe Micron 3400 MTFDKBA512TFH

操作系统：Windows 11 家庭中文版 22H2

实验对象属性

1. 数据库系统配置：PostgreSQL 15
2. 手动编程文件读写程序开发环境：Java JDK17 Maven架构

实验数据

1. filmdb.sql：测试数据文件
2. filmdb2.sql：基于Java程序进行轻微修改的测试数据文件（数据量与filmdb.sql一致）
3. select.sql：针对filmdb的十万次查询操作文件，包含等值查询与模糊查询
4. huge.csv：基于students表结构的数据文件，总计数据量为1亿条（总计2.74G）
5. 1k–1000w.csv 从huge.csv中所截取的前1k~1000w行数据
6. select2.sql：针对students表的十万次主键查询操作文件
7. bank.sql：账户数据库模拟文件。
8. order.sql：订单模拟文件，用于外键测试

实验过程

基本数据操作对比

数据管理系统最为核心的功能便是实现对数据的多种操作，常见的基本数据操作包含数据插入（insert）、数据查询（select）、数据更新（update）等操作。其中，数据查询操作往往最为频繁，应用程序对数据库的查询性能通常具有一定要求，也是数据库技术最常见的优化领域。

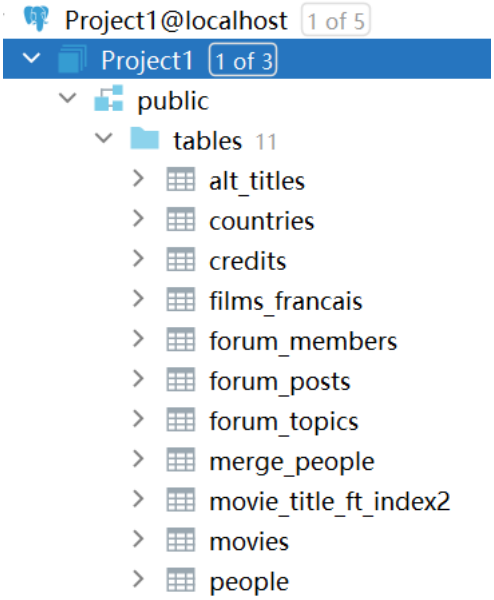
本实验对基本数据操作进行对比试验。实验采用 PostgreSQL数据库系统、Java手动编程（CSV存储格式）、MyDB（自行实现的Java关系型数据库）三种不同系统方式进行数据插入、查询、更新三种基本操作，并对其执行耗时进行对比并分析其结果。实验的具体过程与结果如下。

数据插入操作

数据插入操作是数据管理过程中最重要的操作之一，该操作广泛存在于程序的开发与运行时期。在业务的高速增长期间，可能会出现非常高频繁的数据插入操作，特定业务的实现也可能会需要完成数据的频繁插入操作，同时，数据恢复过程中也需要执行高频的数据插入操作。实验通过模拟数据恢复操作以进行数据的插入效率测试，实验对比PostgreSQL数据库系统、Java手动编程与MyDB系统三种方式导入基于filmdb.sql的数据文件，同时探索对大数据量数据文件huge.csv的多种导入方式，分析不同导入方式之间的效率差异及其原因。

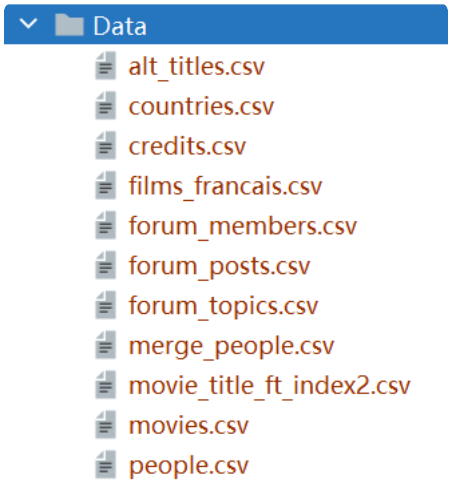
执行filmdb.sql文件以创建并插入数据

1. 采用PostgreSQL数据库系统导入所提供的filmdb.sql文件，记录其消耗时间，实验结果如下



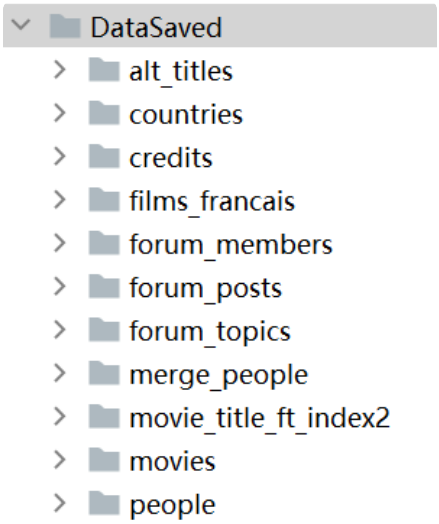
可在数据库可视化工具中查看到表格已被创建

2. 实验采用Java手动编程的方式将filmdb2.sql文件解析并存储至CSV文件中，实验结果如下



可在文件夹中看到存储的csv文件

3. 采用MyDB模拟系统执行filmdb2.sql文件内的代码，实验结果如下



可在文件夹中看到存储的表文件

三种不同方式所消耗的时间对比如下：

系统	PostgreSQL 15	Java 手动编程	MyDB系统
耗时	4.505s	8.293s	12.280s

由上表中我们可以看到，PostgreSQL在执行大批量导入的效率高于Java手动编程与MyDB系统引擎，这可能是由于PostgreSQL系统采用C语言进行开发，C-API的文件IO操作效率往往高于Java原生的文件操作API。同时，java的解释性语言特性使得java程序需在JVM虚拟机中解析class文件，相较于C程序直接编译为本地机器码执行，其运行的固有开销本身便大于C程序。同时，filmdb.sql文件运行过程开启了事务，相较于Java程序每次对文件进行写入，PostgreSQL将在事务结束后再将数据刷入磁盘，从而避免了频繁的I/O操作，关于这一技术的细节将在后文中详细阐述。比较Java手动编程读写与MyDB系统，由于MyDB系统架构更为复杂，其需要花费额外的时间实现对sql语句解析，其也存在更多的虚函数调用，因此Java手动编程文件读写的操作效率高于MyDB系统引擎。

PostgreSQL系统中不同数据插入方式测试

参考所提供的Upload Data.zip文件，我们可以发现多种不同的数据插入方式，速度不同……

1. 使用Awful Loader进行数据导入操作

Awful Loader基于最基本的数据库操作，每条数据的插入操作均为：开启数据库–执行insert语句–关闭数据库，其核心操作如下

Awful Loader 核心操作代码片段 Java

```java
public static void main(String[] args){
    //其他操作
    while ((line = infile.readLine()) != null) {
        //读取每行中的studentid与name
        openDB(/*parmeters*/);
        loadData(studentid, name);
        closeDB();
    }
}
private static void loadData(String studentid, String name) throws SQLException {
    Statement stmt;
     //其他操作
    stmt.execute("insert into students(studentid,name) values('"
        + studentid + "','"+ name.replace("'", "''") + "')");
}
```

由于其插入速度较为缓慢，故只截取了huge.csv的前1000条数据进行插入实验，实验结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin"
1000 records successfully loaded
Loading speed : 14 records/s

Process finished with exit code 0
```

插入速度：14 records/s

2. 使用Very Bad Loader进行数据导入操作

Very Bad Loader在Awful Loader的基础上进行优化，**仅与数据库系统建立了一次连接**。从具体上看，该程序将在插入数据前先建立数据库连接，一次性插入全部数据后再关闭数据库连接，从而减少了建立连接过程的时间消耗，其核心操作如下

Very Bad Loader核心操作代码片段 Java

```java
public static void main(String[] args){
    //其他操作
    openDB(/*parmeters*/);
    while ((line = infile.readLine()) != null) {
        //读取每行中的studentid与name
        loadData(studentid, name);
    }
    closeDB();
}
//loadData函数同Awful Loader
```

实验截取huge.csv前10万条数据进行插入实验，实验结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\
100000 records successfully loaded
Loading speed : 8949 records/s
```

插入速度：8,949 records/s

3. 使用Bad Loader进行数据导入操作

Bad Loader采用**预编译sql语句**的方法，通过预编译insert插入语句，程序在每次进行插入的过程种只需要传递参数，从而避免了对每条数据均执行sql语句解析与执行计划制定的过程，提高了插入效率，其核心操作如下

Bad Loader核心操作代码片段 Java

```java
private static PreparedStatement  stmt = null;//预编译sql语句
public static void main(String[] args){
    //其他操作
    openDB(/*parmeters*/);
    while ((line = infile.readLine()) != null) {
        //读取每行中的studentid与name
        loadData(studentid, name);
    }
    closeDB();
}
private static void openDB(/*parmeters*/) {
    //与postgresql服务器建立连接
    stmt = con.prepareStatement("insert into students(studentid,name)"
                                    + " values(?,?)");
}
private static void loadData(String studentid, String name){
    //其他操作
    stmt.setString(1, studentid);
    stmt.setString(2, name);
    stmt.executeUpdate();
}
```

实验截取huge.csv前10万条数据进行插入实验，实验结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.ex
100000 records successfully loaded
Loading speed : 12134 records/s

Process finished with exit code 0
```

插入速度：12,134 records/s

4. 使用Average Loader进行数据导入操作

Average Loader在Bad Loader的基础上进行优化，通过**开启事务**，待所有数据全部读取并操作完成后再统一将数据写入磁盘中，从而避免了主存储器（RAM）与外部磁盘存储器之间进行频繁的写入操作，提高了插入效率，其核心代码如下

Average Loader核心操作代码片段 Java

```java
private static Connection con;
public static void main(String[] args){
    //其他操作
    openDB(/*parmeters*/);
    while ((line = infile.readLine()) != null) {
        //读取每行中的studentid与name
        loadData(studentid, name);
    }
    con.commit();
    closeDB();
}
private static void openDB(/*parmeters*/) {
    //与postgresql服务器建立连接
    con.setAutoCommit(false);
    //其余部分同Bad Loader
}
//loadData函数同Bad Loader
```

实验截取huge.csv前10万条数据进行插入实验，实验结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.
100000 records successfully loaded
Loading speed : 24061 records/s

Process finished with exit code 0
```

插入速度：24,061 records/s

5. 使用Good Loader进行数据导入操作

Good Loader采用**批量处理方法**，通过将读取的多条数据打包批量发送至postgresql服务器，避免了频繁的网络传输，从而提高了插入运行效率，其核心代码如下

Good Loader核心操作代码片段 Java

```java
private static final int  BATCH_SIZE;
private static Connection con;
public static void main(String[] args){
    //其他操作
    int cnt = 0;
    openDB(/*parmeters*/);
    while ((line = infile.readLine()) != null) {
        //读取每行中的studentid与name
        loadData(studentid, name);
        cnt++;
        if (cnt % BATCH_SIZE == 0) {
            stmt.executeBatch();
            stmt.clearBatch();
        }
    }
    con.commit();
    closeDB();
}
//openDB函数同Average Loader
private static void loadData(String studentid, String name){
    //其他操作
    stmt.setString(1, studentid);
    stmt.setString(2, name);
    stmt.addBatch();
}
```

实验截取huge.csv前10万条数据进行插入实验，实验结果如下

a. 设置BATCH_SIZE=10

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe"
100000 records successfully loaded
Loading speed : 88809 records/s

Process finished with exit code 0
```

b. 设置BATCH_SIZE=50

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.ex
100000 records successfully loaded
Loading speed : 119760 records/s

Process finished with exit code 0
```

c. 设置BATCH_SIZE=100

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe"
100000 records successfully loaded
Loading speed : 127388 records/s

Process finished with exit code 0
```

d. 设置BATCH_SIZE=500

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.ex
100000 records successfully loaded
Loading speed : 141242 records/s

Process finished with exit code 0
```

e. 设置BATCH_SIZE=1000

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.e
100000 records successfully loaded
Loading speed : 142653 records/s

Process finished with exit code 0
```

f 设置BATCH_SIZE=5000

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe"
100000 records successfully loaded
Loading speed : 138888 records/s

Process finished with exit code 0
```

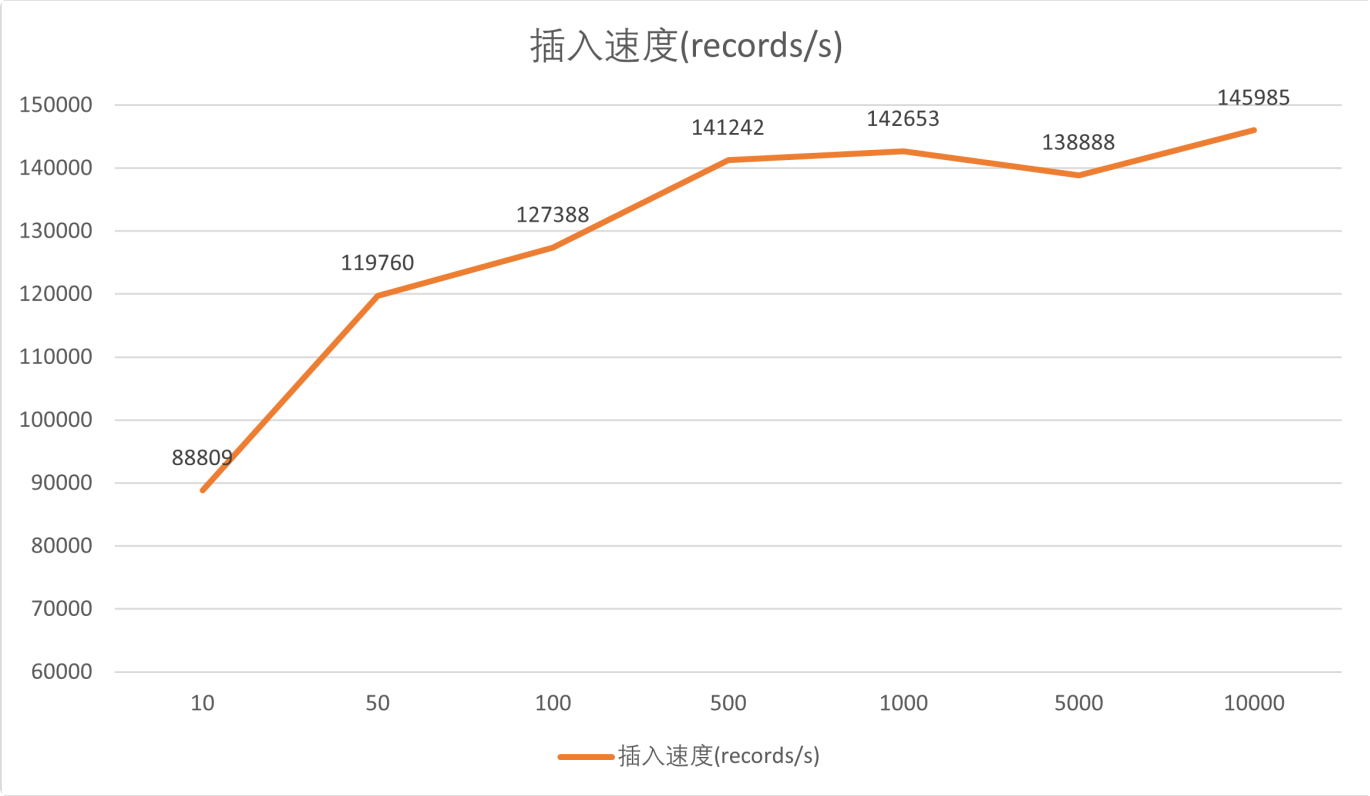
g. 设置BATCH_SIZE=10000

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" "-
100000 records successfully loaded
Loading speed : 145985 records/s

Process finished with exit code 0
```

不同BATCH_SIZE条件下的插入速度如下

BATCH_SIZE	10	50	100	500	1000	5000	10000
插入速度 (records/s)	88809	119760	127388	141242	142653	138888	145985



可以看到，在BATCH_SIZE较小（≤500）时，插入速度随BATCH_SIZE的增大出现明显提升，表明**批量发送多条语句可以有效提高插入速度**。而当BATCH_SIZE较大（≥1000）时，插入速度趋于稳定，**此时BATCH_SIZE的增大对插入速度没有产生明显提升**，表明此时插入速度可能主要受限于插入处理与磁盘I/O速度，提高单数据包数据量对插入速度影响有限。

6. 采用多行插入语句批量进行数据导入操作

在插入语句中，通过将单行插入语句"insert into ... values(?, ?)" 改写为多行插入语句"insert into ... values(?, ?), (?, ?),..."，可以在一次插入语句执行的过程中插入多行数据，从而减少了insert语句的执行次数，提高了插入运行效率。在Postgresql-JDBC中，通过设置reWriteBatchedInserts=true，可以实现上述该写操作。

实验设置BATCH_SIZE=1000并开启reWriteBatchedInserts，截取huge.csv前10万条数据进行插入实验，实验结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java
100000 records successfully loaded
Loading speed : 235849 records/s

Process finished with exit code 0
```

插入速度：235,849 records/s

7. 使用copy关键字进行数据导入操作

copy关键字是Postgresql所支持的一种操作指令，使用copy关键字进行数据导入时可以避免insert语句操作所需要进行的数据格式转换操作，从而显著地提高了插入语句的执行效率，该语句的操作指令如下所示。

copy操作指令 SQL

```
copy students from 'huge.csv' delimiter ',';
```

实验使用huge.csv一亿条数据进行插入实验，实验结果如下

```
                    delimiter ','
21:50:05] 100,000,000 rows affected in 2 m 19 s 281 ms
```

插入速度：717,973 records/s

8. 采用无日志表进行加速

无日志表是Postgresql中一种数据表，当执行写入操作时不会产生WAL日志记录，从而加快了写入速度。但WAL日志对于维持数据完整性十分重要[https://www.postgresql.org/docs/current/sql-createtable.html]，当写入事务夭折时，WAL日志将用于完成事务回滚。因此，无日志表适用于不使用事务模型保障数据完整性，写入操作（insert/update/delete）频繁的数据表。同时，对于需要采用事务模型保障数据完整性的数据表，可能可以先创建Unlogged数据表进行插入操作，待插入完成后再通过代码将日志表切换为logged模式。

无日志表的创建sql语句如下

无日志表创建sql语句 Java

```
create unlogged table students
(
    id        serial primary key,
    studentid varchar(10),
    name      varchar
);
```

a. 使用多行插入语句进行数据导入操作

实验设置BATCH_SIZE=1000，使用huge.csv中前10万条数据进行插入实验，实验结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" "
100000 records successfully loaded
Loading speed : 270270 records/s

Process finished with exit code 0
```

插入速度：270,270 records/s

可以看到，相较于有WAL日志表，采用**无日志表可以提升约10%的insert插入效率**，其性能提升较为显著。

b. 使用copy关键字进行数据导入操作

使用copy关键字进行导入时，先创建unlogged表，再执行如下指令将数据表设置为logged表

sql语句 Java

```
alter table students set logged ;
```

实验使用huge.csv一亿条数据进行插入实验，实验结果如下

```
delimiter ','
22:10:58] 100,000,000 rows affected in 2 m 19 s 585 ms
```

插入速度：716,409 records/s

```
HugeData.public> alter table students set logged
[2023-10-06 22:19:38] completed in 2 m 9 s 539 ms
```

可以看到采用无日志表并未对插入速度产生明显提升，这是由于copy操作过程并不需要产生大量的日志记录。除此之外，unlogged表与logged表之间的切换也需耗费大量的时间执行。因此，通过**先创建unlogged数据表，使用copy关键字进行插入再将表切换为logged模式的操作并不能有效提高数据插入速度**。

9. 关闭索引与约束

PostgreSQL为保障数据一致性与提高查询效率，采用了索引与约束限制等技术对数据进行额外的处理操作，其细节将在下文中详细阐述。数据库维护数据的索引与约束的过程需要进行一定的开销，在数据导入恢复的过程中，可以暂时关闭数据库的约束与索引功能，从而提高数据的导入效率。实验将对此进行探究

a. 关闭id主键约束（主键约束同时会自动创建索引），执行如下代码以创建无自增主键的students表

sql创建代码（无自增主键） Java

```
create table students
(
    id        serial,
    studentid varchar(10),
    name      varchar
);
```

执行上述copy语句进行数据导入操作，其执行结果如下

```
HugeData.public> copy students from 'C:\A\huge.sql'
                 delimiter ','
[2023-10-06 01:57:05] 100,000,000 rows affected in 1 m 38 s 994 ms
```

插入速度：1,010,162 records/s

b. 进一步关闭students_id_seq自增约束，执行如下创建代码

sql创建代码（无自增主键） Java

```
create table students
(
    id        int,
    studentid varchar(10),
    name      varchar
);
```

执行上述copy语句进行数据导入操作，其执行结果如下

```
delimiter ','
 100,000,000 rows affected in 1 m 20 s 766 ms
```

插入速度：1,238,145 records/s

c. 将数据表恢复至原先包含自增约束与主键约束的结构，执行如下sql代码

sql代码如下 SQL

```sql
begin ;
create sequence students_id_seq start 100000001;
alter table students alter column id set default nextval('students_id_seq')
,
    add primary key(id);
commit;
```

代码执行结果如下

```
begin
:26] completed in 0 ms
create sequence students_id_seq start 100000001
:26] completed in 2 ms
alter table students alter column id set default nextval('students_id_seq'),
    add primary key(id)
':27] completed in 1 m 1 s 181 ms
commit
```

总计耗时（无主键与自增约束插入+恢复主键与自增约束）：141.947s

我们可以看到，采取先关闭索引与约束，数据导入后再创建的总共耗时与不关闭索引与约束进行导入的**总计耗时（139.281s)较为接近**，这是由于先关闭索引与约束的导入方式只是将数据库维护索引与约束的过程推迟到了导入操作完成以后，但该种方式能够有效地减少导入过程的耗时。因此，**如在现实开发运行过程中需尽快恢复数据库中所存储的大量数据，则可以采用此种方式加快导入操作。**

数据查询操作

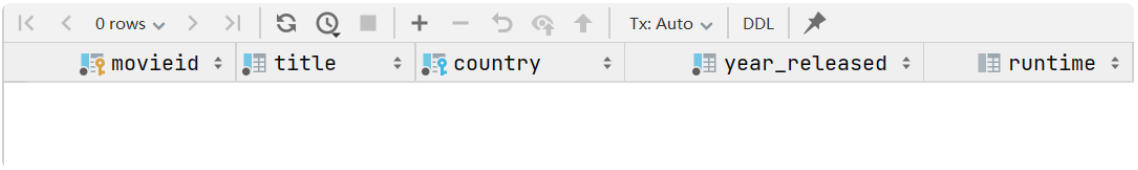
查询movies表中title字段包含"XXX"的元组

1. 在PostgreSQL控制台中执行如下语句

SQL语句1 SQL

```sql
select * from movies where title like '%XXX%';
```

执行结果如下：



2. 采用Java手动编程的方式查询movies数据表中的title字段包含"XXX"的元组，执行结果如下：

```
movieid     title       country     year_released       runtime
```

总计耗时：27毫秒

3. 采用MyDB模拟系统执行 SQL语句1 ， 执行结果如下：

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" ...
Welcome to MyDB! Input SQL Command to control the system.
select * from movies where title like '%XXX%';
movieid     title       country     year_released       runtime
```

耗时：145毫秒

三种不同方式所消耗时间对比如下：

系统	PostgreSQL 15	Java 手动编程	MyDB系统
耗时	0.183s	0.027s	0.145s

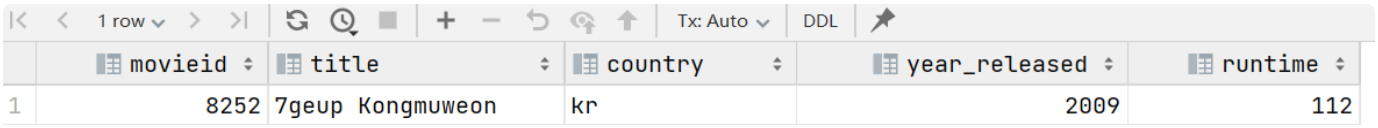
对movies表中主键数据进行查询操作

1. 在PostgreSQL控制台中执行如下语句

SQL语句2 SQL

```sql
select * from movies where movieid=8252;
```

执行结果如下：



	movieid	title	country	year_released	runtime
1	8252	7geup Kongmuweon	kr	2009	112

2. 采用Java手动编程的方式查询movies数据表中的movieid字段值为8252的元组，执行结果如下：

```
movieid     title       country     year_released       runtime
8252    '7geup Kongmuweon'      'kr'    2009    112
```

总计耗时：27毫秒

3. 采用MyDB模拟系统执行 SQL语句2 ， 执行结果如下：

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" ...
Welcome to MyDB! Input SQL Command to control the system.
select * from movies where movieid=8252;
movieid     title       country     year_released       runtime
8252    7geup Kongmuweon    kr      2009    112

耗时：162毫秒
```

三种不同方式所消耗时间对比如下：

系统	PostgreSQL 15	Java 手动编程	MyDB系统
耗时	0.039s	0.027s	0.162s

我们可以发现，相较于查询movies表中title字段包含"XXX"的元组的操作，PostgreSQL对主键查询操作操作的运行效率显著较高，这是由于主键查询采用了主键索引的技术优化了查询效率。与此同时，对于第一组实验的字段模糊查询，Java手动编程程序的执行效率快于PostgreSQL系统的执行效率，而第二组实验两者的运行速度较为接近，这可能是由于单条语句执行时间较短所带来的误差，但同时查询资料我们发现，传统的火山模型架构由于在执行sql语句过程中存在大量的虚函数调用，其执行性能也可能低于相同条件下对数据的直接操作[1]。

对movies表中不同数据进行频繁查询

为更好地模拟现实数据库系统使用过程中可能出现的高频繁查询，同时降低因单条查询语句执行时间较短所可能带来误差，本实验选择了数据量较多的credits,movies,people,alt_titles四张数据表，通过编写java程序针对所选数据表随机生成了10万条SQL查询语句，查询语句中包含主键查询与模糊查询。随后分别使用三种不同系统进行查询操作，为避免结果输出至控制台过程中可能会出现的性能损耗，三组实验均关闭大部分结果输出。

1. 在PostgreSQL系统中执行select.sql，执行结果如下（仅打印最后一条查询结果）：

	movieid [PK] integer	title character varying (100)	country character	year_released integer	runtime integer
56	6911	Cry Freedom	za	1987	157
57	7036	The Gods Must Be Crazy	za	1980	109
58	7056	Sione's Wedding	nz	2006	92
59	7293	What We Do in the Shadows	nz	2014	85
60	7339	Já, Olga Hepnarová	cz	2016	105
61	7359	Never Say Die	nz	1988	90
62	8734	The League of Extraordinary Gentlemen	cz	2003	110
63	8885	The Tunnel	za	2009	25
64	8982	Sibirskij tsirjul'nik	cz	1998	180

Total rows: 64 of 64 | Query complete 00:02:31.700

2. 采用Java手动编程方式对select.sql进行解析并将参数传入查询函数，执行结果如下：

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" "-ja
总计耗时：378090毫秒

Process finished with exit code 0
```

3. 采用MyDB模拟系统执行 select.sql，执行结果如下（每完成一万次查询输出一条日志）：

```
FilmDBSelect
第40000条查询完成，耗时：313247ms
第50000条查询完成，耗时：383532ms
第60000条查询完成，耗时：453458ms
第70000条查询完成，耗时：523818ms
第80000条查询完成，耗时：594303ms
第90000条查询完成，耗时：664259ms
第100000条查询完成，耗时：734398ms
总耗时：734399ms

Process finished with exit code 0
```

系统	PostgreSQL 15	Java 手动编程	MyDB系统
耗时	151.700s	378.090s	734.399s

可以发现，PostgreSQL系统在执行大量查询操作时具有较为明显的性能优势，这是由于PostgreSQL系统在查询过程中采用了多种优化手段，通过降低i/o次数与访问数据量，有效地提高了数据库的执行效率。为探究数据库系统所采用的技术及其效果，研究分析了大数据量表下数据库为提升数据操作速度所采取的优化技术，并设计了大数据量表查询测试对其进行验证。

大数据量表查询测试

对于大数据量表的数据操作，数据库系统采用了多种方式以提升数据操作效率，其中主要采用了数据分页，开启缓存、建立索引结构的方式减少磁盘I/O操作与搜索数据量，从而提升数据库的数据操作效率。研究对数据库所采用的这三种方式的技术细节进行了分析，并设计实验研究其实际效果。

数据分页

相较于Java-CSV手动编程方式下对表创建唯一的文件并在其中进行读写，PostgreSQL数据库将数据以页文件的方式分块存储，进入PostgreSQL数据库表文件存储路径下，我们可以看到，文件夹下存在大量大小均为8k的页文件。



采取数据分页的策略是由物理存储设备与操作系统决定的，现行的linux/windows等操作系统均是以磁盘块为单位进行数据管理，操作系统将物理磁盘所划分的多个扇区合并为块作为读写操作的最小单位。现有的操作系统最常用的磁盘块大小为4kb，即使数据文件仅占一个字节大小，文件系统仍需为其分配最少为4kb的大小进行存储，同时也存在2kb或1kb的磁盘块大小。PostgreSQL系统参考现有操作系统大的磁盘块小，将存储文件的数据块大小默认设为8kb，保证存储文件大小可以恰好分为两个或其它整数个磁盘块，避免出现数据文件无法被恰分而需开辟新磁盘块的情况。同时，相较于将所占用内存大小设为更大的值，选择8kb的大小也能避免页面空间的浪费。

若采用类似于手动编程将数据存储至单一文件的方式，则该文件可能会因数据量的增加而需占用更大的存储空间，这将使操作系统不得不为该文件分配更多的磁盘块，这些磁盘块很可能是非连续的，并且该过程可能会产生大量的磁盘碎片，从而降低对该文件I/O操作的效率，而PostgreSQL系统所采用的固定页大小能有效地避免产生大量的磁盘碎片与减少单文件占用磁盘块的不连续情况，从而避免了上述情形的发生。

最后，PostgreSQL系统采用了索引结构等优化方式，使对表进行查询等操作过程并不需要真正对数据表文件的所有数据行进行访问，将数据分散至不同的数据页能够使PostgreSQL系统只选择所需要的部分行数据进行加载，从而避免了对大文件的I/O操作。

综合以上分析，我们发现，**采用数据分页的方式能够提升对数据读取过程中的I/O操作效率**，PostgreSQL将页文件大小设置为8kb**有效平衡了I/O效率与存储空间大小**。实质上，对于数据库较大的情形，在操作系统支持huge_page分配大磁盘页的条件下，还可以将PostgreSQL数据页设置为大数据页，从而进一步提高对I/O操作效率。

缓存与缓存替换策略

缓存指的是在数据交换过程中为快速操作而专门开辟出的一块缓冲区域。由于在计算机系统中，内存（RAM）是计算机用于暂时存储数据和程序信息的主要区域，其读写速度往往高于对磁盘的读写速度，但内存具有容量体积小，断电后数据无法保存的特点，不适宜用于数据库数据的持久化存储。磁盘则具有断电数据不易失，存储容量大的特点，适合用于存放数据库数据，但其读写速度相较于内存较慢，I/O操作的耗时较长。因此，数据库系统往往采用缓存策略，通过在RAM上开辟一段缓存空间，将磁盘中常用的数据读取至缓存中，再次访问数据时可以直接从缓存区中进行快速读写，从而提高了数据库操作速度。

实验对数据库的缓存有效性进行测试，同时修改Java-CSV手动编程程序将数据表预加载至缓存区中，探索缓存技术对提升数据操作速度的可行性，实验操作与结果如下。

1. 对数据库缓存有效性进行测试，测试过程如下
    a. 重启PostgreSQL服务以清空缓存，对PostgreSQL数据库系统的同个访问连接中多次执行如下查询语句，观察数据获取时间差异

测试查询语句 SQL

```sql
select * from people where born=1984;
```

测试结果如下

```
ic> select * from people where born=1984
3:55:31] 179 rows retrieved starting from 1 in 197 ms
ic> select * from people where born=1984
3:56:28] 179 rows retrieved starting from 1 in 36 ms
ic> select * from people where born=1984
3:56:42] 179 rows retrieved starting from 1 in 28 ms
ic> select * from people where born=1984
4:00:53] 179 rows retrieved starting from 1 in 39 ms
ic> select * from people where born=1984
4:00:57] 179 rows retrieved starting from 1 in 21 ms
```

我们可以看到，在执行同一条select语句中，后续执行所耗费的时间显著快于第一次执行所耗费的时间，这可能受到数据库缓存机制、操作系统缓存机制等多种因素的影响。

    b. 再次重启PostgreSQL服务，多次执行如下语句查看PostgreSQL执行SQL语句的详细信息

SQL

```sql
explain (analyze,buffers) select * from people where born=1984;
```

第一次执行时结果如下

```
Seq Scan on people  (cost=0.00..364.16 rows=200 width=27)
  Filter: (born = 1984)
  Rows Removed by Filter: 16310
  Buffers: shared read=134
```

后续执行时结果如下

```
Seq Scan on people  (cost=0.00..364.16 rows=200 width=27)
  Filter: (born = 1984)
  Rows Removed by Filter: 16310
  Buffers: shared hit=134
```

可以明显地看到，虽然每一次执行该语句都需要进行全表扫描操作，但第一次执行时，数据库必须从磁盘中将数据块读取至数据库缓存中（即在执行计划中提示read），而在后续执行过程中，数据库直接在

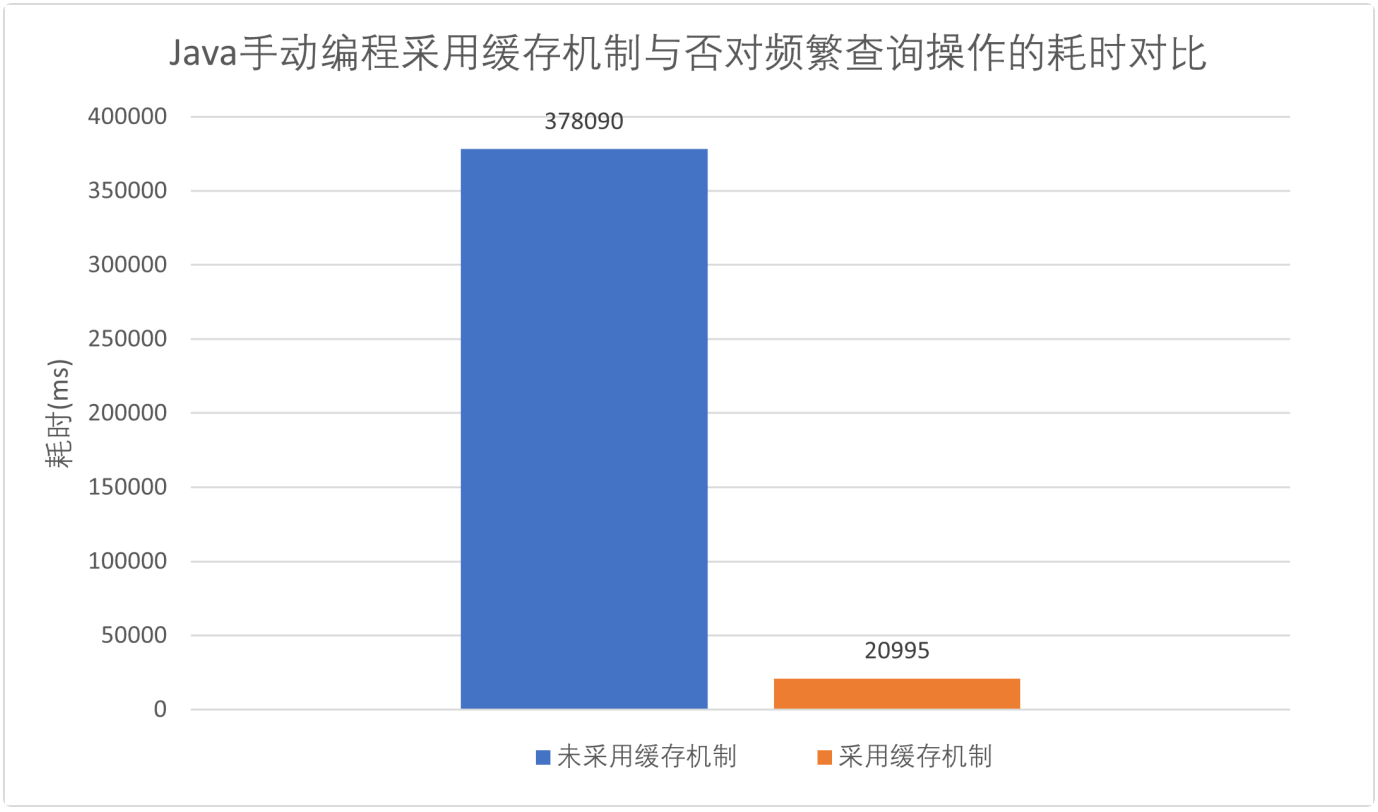
缓存区中命中了上次读取操作所缓存的数据（即在执行计划中提示hit）。通过采用将常用数据进行缓存存储的技术，数据库系统可以在一定程度上提升了数据查询等过程中的操作效率。

2. 实验修改Java-CSV手动编程程序，通过在每次读取数据表后将数据表缓存至缓冲区中，在每次对表执行select语句时先在缓存区中查看是否存在该表，若存在则不对磁盘进行I/O读取操作，从而模拟了现实数据库系统的缓存机制。对该缓存频繁查询程序执行select.sql语句，对比未采用缓存机制每次对数据表进行读取的操作方式，实验结果如下。

```
LargeQueryCache
"C:\Program Files\Java\jdk-17.0.4.1\l
第10000条查询完成，耗时：2426ms
第20000条查询完成，耗时：4104ms
第30000条查询完成，耗时：6101ms
第40000条查询完成，耗时：8039ms
第50000条查询完成，耗时：10020ms
第60000条查询完成，耗时：12375ms
第70000条查询完成，耗时：14534ms
第80000条查询完成，耗时：16729ms
第90000条查询完成，耗时：18941ms
第100000条查询完成，耗时：20995ms
总计耗时：20995毫秒

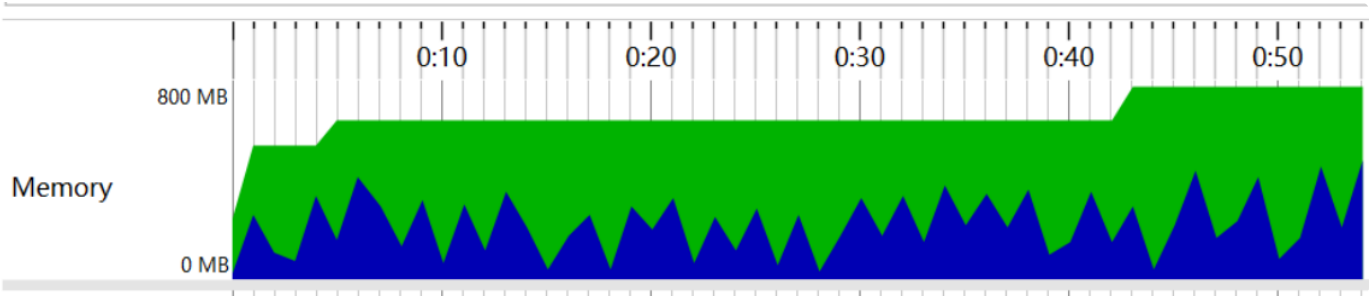
Process finished with exit code 0
```

操作方式	未采用缓存机制	采用缓存机制
耗时	378.090s	20.995s



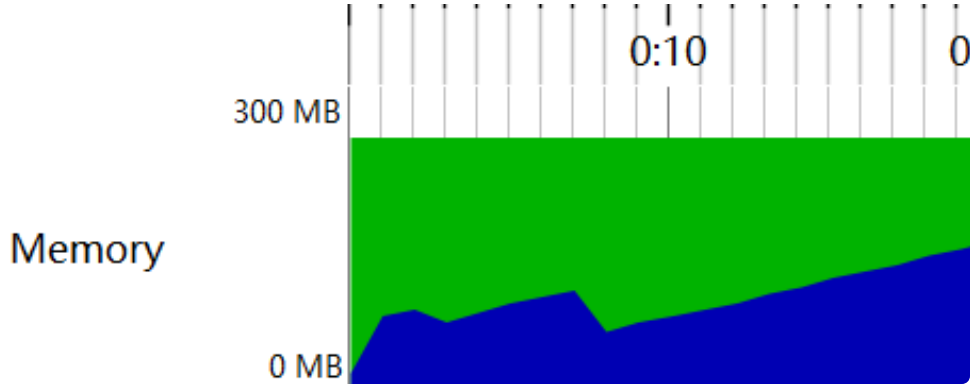
我们可以看到，修改后的采用缓存机制的代码十分高效地完成了查询操作，相较于未采取缓存机制速度提升超10倍。这表明采用缓存机制避免对磁盘的频繁IO操作能够十分有效地提升数据库的数据操作效率。

对Java程序运行时的内存情况进行监控，我们可以从下图中明显的看到，对于未采用缓存机制的Java程序，JVM虚拟机内存中程序所使用空间（蓝色）变化十分剧烈，同时GC垃圾收集器不断地回收着读表查询后又释放的表数据。



未采用缓存机制内存变化情况

而对于采用缓存机制的Java程序，JVM虚拟机内存中程序所使用空间较为稳定，GC垃圾回收器没有进行频繁回收操作，采用缓存机制也可以避免频繁的资源读取与释放，在一定程度上提升读取效率，尤其是对于JVM虚拟机这类采用GC垃圾回收器会回收无用对象的程序，可以降低GC的回收负担，减少系统资源的占用。



采用缓存机制内存变化情况

虽然将数据加载至缓存中能有效提高数据库系统的运行效率，但数据库系统的缓存空间并非无限，随着数据库的运行，数据库内部的缓存空间可能被完全占用，因此需要移除缓存数据中那些不常被使用的数据。缓存技术中最常使用的缓存淘汰算法为LRU算法，它将会记录缓存数据中最久未被使用的数据并在缓存区已满时淘汰这些数据。而PostgreSQL系统则在LRU算法的基础上进行优化设计了Clock算法，通过标记页数据是否被修改，从而尽量在数据已满时优先淘汰未被修改的数据，尽量避免因淘汰修改后数据而必须要进行的数据刷入磁盘操作[2]，从而减少了对磁盘的IO操作，提高了数据库系统的读写效率。

与此同时，由上文所述，PostgreSQL数据库系统采用了数据分页的方式，避免了访问表的过程中需要将全表数据一并加载至RAM中的操作，降低了磁盘I/O操作负担，结合缓存更新算法能够有效避免数据量超出RAM内存限制。实验通过对Java-CSV手动编程程序进行大表查询操作进行对比测试。

1. 采用huge.csv中的数据作为students表数据，尝试对students表进行select操作，操作过程如下

   a. 在PostgreSQL数据库中执行如下语句

SQL语句4 SQL

```sql
select * from students where name='龙志';
```

执行结果如下

	id	studentid	name
1	2529	2529	龙志
2	11353	11353	龙志
3	43149	43149	龙志
4	72783	72783	龙志
5	88542	88542	龙志
6	101928	101928	龙志
7	121360	121360	龙志
8	138790	138790	龙志
9	189694	189694	龙志
10	191062	191062	龙志

采用explain语句查看该语句的执行情况，我们可以看到，数据库仅在buffers中命中了部分数据，剩余的数据则仍需从磁盘中进行读取

```
Buffers: shared hit=260 read=313918
->  Parallel Seq Scan on students  (cost=0.00..366065.70 rows=264 width=20) (actua
      Filter: ((name)::text = '龙志'::text)
      Rows Removed by Filter: 3333143
      Buffers: shared hit=260 read=313918
```

   b. 在Java-CSV手动编程中执行select语句，执行结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" -agentlib:jdwp=transport=dt_socket,a
Connected to the target VM, address: '127.0.0.1:4565', transport: 'socket'
Exception in thread "main" java.lang.OutOfMemoryError Create breakpoint : Java heap space
    at java.base/java.io.BufferedReader.readLine(BufferedReader.java:362)
    at java.base/java.io.BufferedReader.readLine(BufferedReader.java:396)
    at CSVHandler.readFile(CSVHandler.java:50)
    at LargeDataQuery.main(LargeDataQuery.java:11)
Disconnected from the target VM, address: '127.0.0.1:4565', transport: 'socket'
```

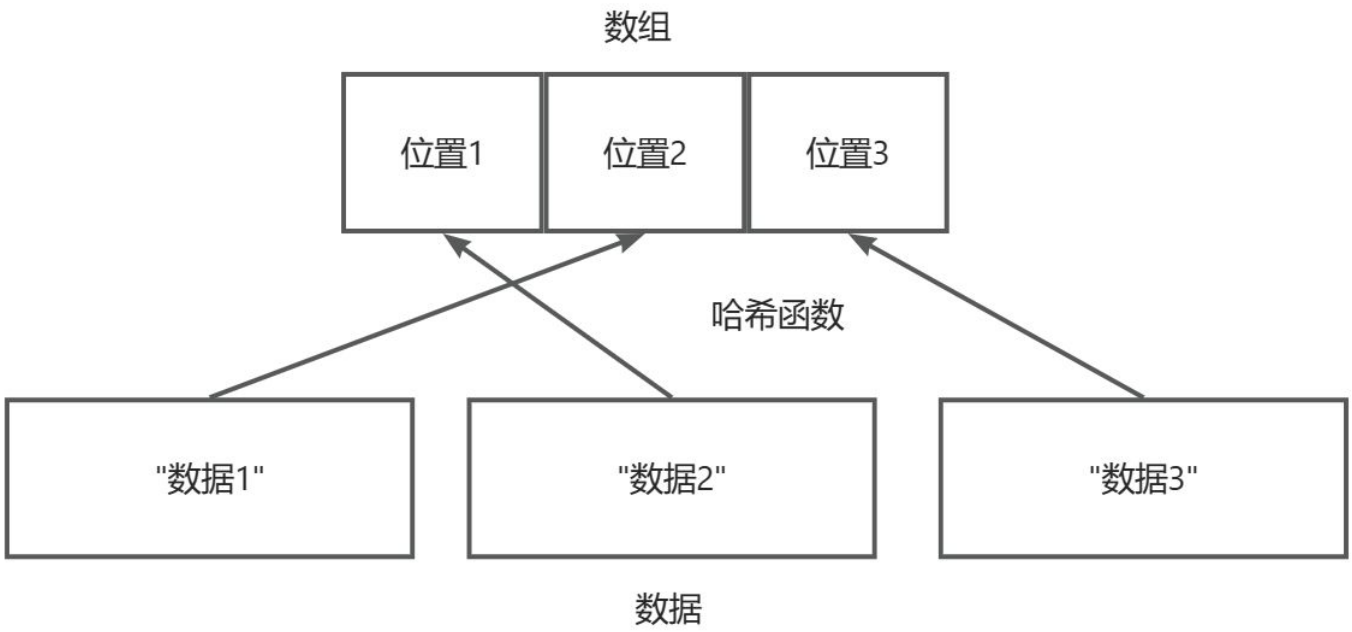
执行过程中抛出了内存溢出错误，这是由于JVM虚拟机中默认的最大内存大小为2GB，而students表中数据为约2.6GB，超出了JVM堆的最大存储大小，将其数据一并读取至内存的操作过程将会造成内存溢出。

综合上述实验我们发现，数据库系统所采用的**缓存技术能够在有效地缓解数据处理操作过程中的磁盘IO性能瓶颈**，从而大大地提高了数据库系统的运行效率。同时，**缓存技术也避免了频繁的资源占用与释放**，能在一定程度上提升整体系统的运行效率。同时，数据库系统**采用了一系列的缓存淘汰算法以在缓存已满时有选择地淘汰部分数据，从而在保证不发生内存溢出的同时近最大可能提升缓存命中的概率**，从而提升数据库的运行效率。

索引结构

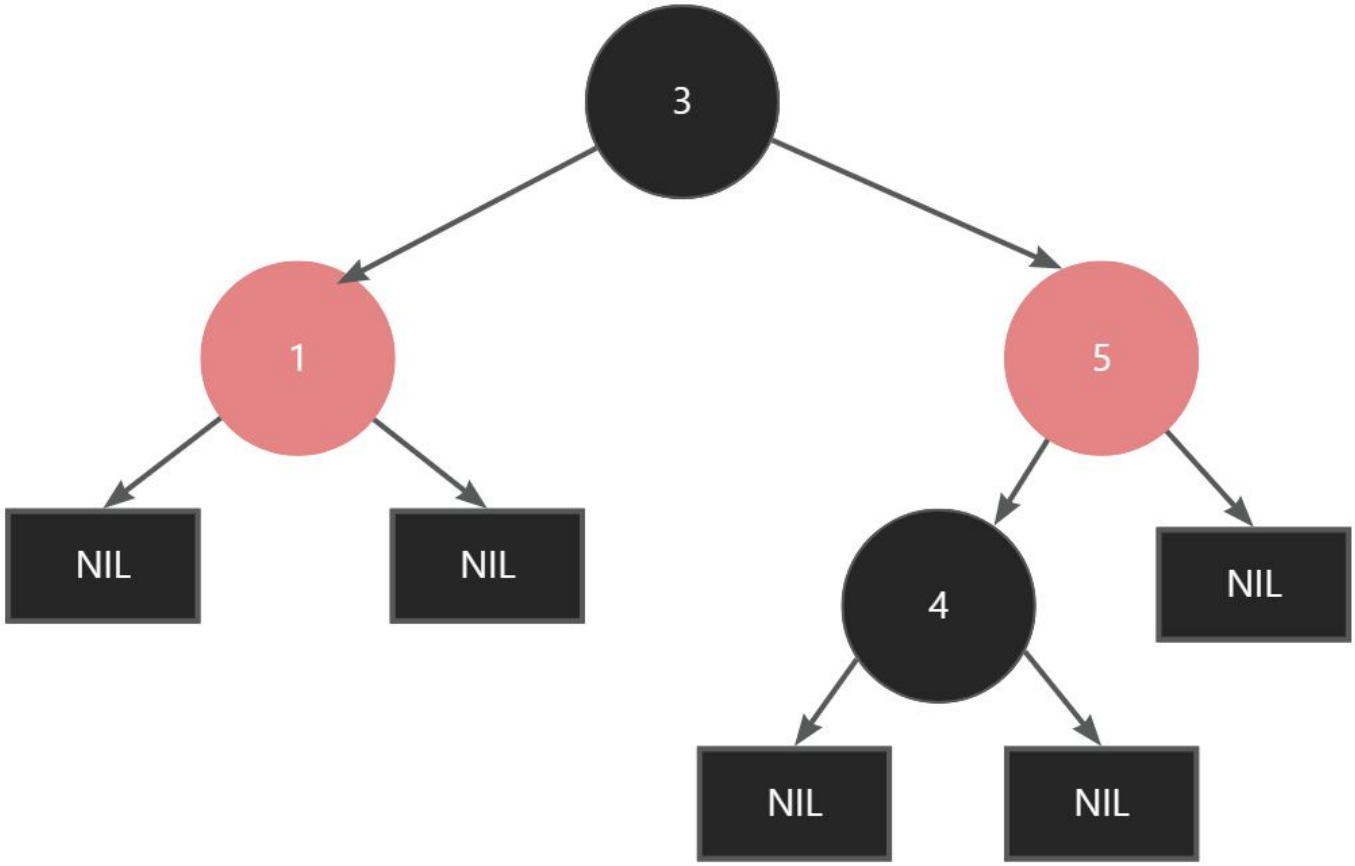
为提高数据库系统的检索效率，数据库系统往往采用索引技术减少所需访问的数据条目。索引常采用一种数据结构管理数据间关系，从而在算法层面提高查找过程的运行效率。数据库系统索引技术所最常用的数据结构有哈希表、红黑树、B树与B+树。

对于哈希结构索引，数据库系统将通过算法计算数据的哈希值并存储至特定的位置，实现对数据与数据存储位置的映射过程。当需要查找某条数据时，可以直接通过求出该数据哈希值找到其存储的位置，其结构如下图所示



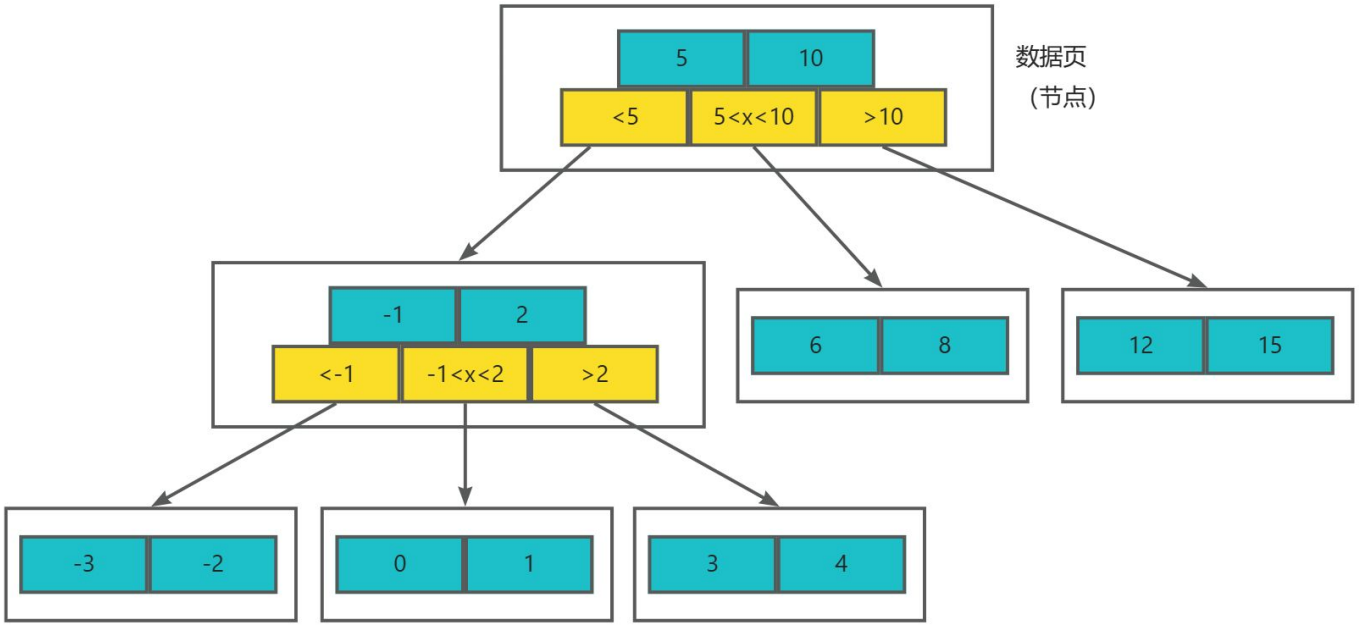
采用哈希结构索引的插入、删除与等值检索的时间复杂度均为O(1)，等值查找速度往往十分迅速，但当数据量大时，可能会出现哈希碰撞的问题而降低检索速度，同时，采用哈希索引结构难以实现对范围查找等复杂查找方式的支持，数据库系统可能会采用顺序查找的方式执行此类查找的操作过程，从而显著降低了查询效率，最后，哈希索引往往需要一次性将全部数据加载至RAM中以便于碰撞查找，如果数据量过大无法一次加载，则需要分批次进行加载并进行多次哈希查找，其实现较为复杂且会降低查找效率。

除采用哈希表外，数据库系统也可能采用红黑树实现数据的索引结构。红黑树是一种自平衡的二叉搜索树，通过限制二叉树所必须需满足的特定性质，红黑树能够保证二叉树不会退化为链表结构。同时，红黑树也能有效减少平衡过程中需要的旋转次数。红黑树的大致结构如下图所示



红黑树能够保证对数据的查询、插入与删除等操作均O(logn)复杂度，从而有效降低了查询操作所需的耗时，同时，红黑树能够容易地分批次将索引数据加载至内存中，支持数据量庞大条件下的索引操作。但当数据量较大时，红黑树高度往往会非常高，往往会造成数据无法一次读入，需进行频繁的磁盘IO操作，因此，数据库系统往往使用B/B+树以实现数据的索引结构。

B/B+树均是多路搜索树，相较于红黑树每个节点只有两个叶子节点，B/B+树的每个节点下包含多个叶子节点，其每个节点的数据均是有序排布的。对于B树的查找工作将从根节点开始，对节点内部有序数据进行二分查找，若命中数据则直接返回，若该数据属于本节点中所存储的范围指针，则进一步进入该指针所指向的子节点进行查找。其大致结构如下，其中蓝色部分为存放数据的数组，而黄色部分则为指向下一节点的指针



B+树在B树的结构基础上进行调整，通过将数据全部放置在叶子节点，强制仅有在叶子节点才能命中数据，同时在叶子节点之间用指针构成双向链表。B/B+树均采用多路搜索的方法减少了磁盘IO的次数，其插入、删除与查询时间复杂度均为O(logn)，能够有效提升查询的速度。同时相较于B树，B+树将数据存放至叶子节点能够更加便于对数据的范围查询。

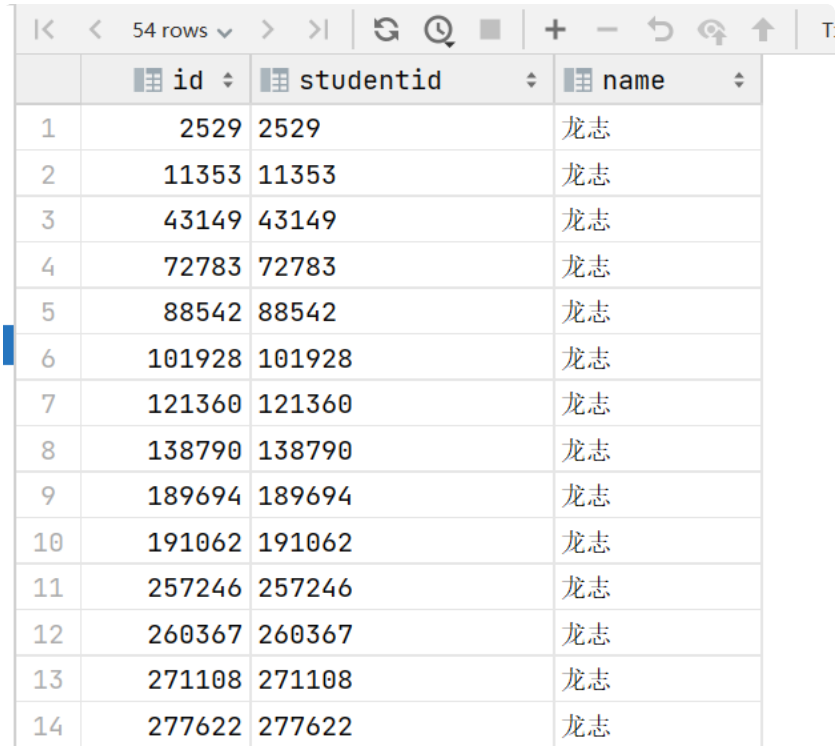
PostgreSQL15系统已经支持对数据哈希索引与B树索引两种索引结构。本实验分别使用huge.csv前100万条数据，前1亿条数据进行查询实验。通过对比Postgresql在不对检索字段建立索引索引、对检索字段建立哈希索引、B树索引三种方式，同时对比采用Java-CSV手动编程的方式数据进行全表遍历检索与MyDB系统CSV引擎下进行全表遍历检索，探究索引结构对加快数据检索操作的有效性。实验具体过程与结果如下

1. 分别对students表100万数据量、1亿数据量情况进行查询实验，执行简单等值查询语句

SQL语句4 SQL

```sql
select * from students where name='龙志';
```

1.1. 使用PostgreSQL系统不针对name字段创建任何索引，执行SQL语句4进行查询，查询结果如下



54 rows

	id	studentid	name
1	2529	2529	龙志
2	11353	11353	龙志
3	43149	43149	龙志
4	72783	72783	龙志
5	88542	88542	龙志
6	101928	101928	龙志
7	121360	121360	龙志
8	138790	138790	龙志
9	189694	189694	龙志
10	191062	191062	龙志
11	257246	257246	龙志
12	260367	260367	龙志
13	271108	271108	龙志
14	277622	277622	龙志

100万数据量查询耗时：75 ms

1亿数据量查询耗时：1166ms

1.2. 在PostgreSQL系统中为name字段建立哈希索引，执行如下SQL语句

SQL语句-创建哈希索引 SQL

```sql
create index students_name_idx on students using hash(name);
```

随后执行SQL语句4进行查询，查询结果同1.1

100万数据量查询耗时：8ms

1亿数据量查询耗时：392ms

1.3. 在PostgreSQL系统中为name字段建立B树索引，执行如下SQL语句

SQL语句-创建B树索引 SQL

```sql
create index students_name_idx on students using btree(name);
```

随后执行SQL语句4进行查询，查询结果同1.1

100万数据量查询耗时：6ms

1亿数据量查询耗时：7ms

1.4. 采用Java手动编程的方式执行SQL语句4进行查询，查询结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.e
Connected to the target VM, address: '127.0.0.
id      studentid       name
2529    2529    龙志
11353       11353       龙志
43149       43149       龙志
72783       72783       龙志
88542       88542       龙志
101928      101928      龙志
121360      121360      龙志
138790      138790      龙志
189694      189694      龙志
191062      191062      龙志
257246      257246      龙志
260367      260367      龙志
271108      271108      龙志
277622      277622      龙志
287384      287384      龙志
288856      288856      龙志
293200      293200      龙志
295618      295618      龙志
```

100万数据查询耗时：287ms

在执行1亿数据量查询操作时，由于数据量较大，发生了同缓存替换策略部分实验所出现的堆溢出错误。

1.5. 采用MyDB系统CSV存储引擎执行SQL语句4进行查询，查询结果如下

```
Welcome to MyDB! Input SQL Command to control the system.
select * from students where name='龙志';
Executing sql command: select * from students where name='龙志';
id      studentid       name
2529    2529    龙志
11353       11353       龙志
43149       43149       龙志
72783       72783       龙志
88542       88542       龙志
101928      101928      龙志
121360      121360      龙志
138790      138790      龙志
189694      189694      龙志
191062      191062      龙志
257246      257246      龙志
260367      260367      龙志
271108      271108      龙志
```

100万数据查询耗时：1543ms

不同系统与索引情况下单条等值查询耗时汇总

	单表100万数据	单表1亿数据
PostgreSQL 无索引	75ms	12794ms
PostgreSQL 哈希索引	8ms	392ms
PostgreSQL B树索引	6ms	7ms

Java手动编程	287ms	/
MyDB CSV存储引擎	1543ms	/

结果分析

通过执行explain指令，我们生成了一百万数据量下上述SQL语句的查询计划

无索引实验组SQL语句执行计划如下

```
Gather  (cost=1000.00..12619.73 rows=64 width=19)
  Workers Planned: 2
  ->  Parallel Seq Scan on students  (cost=0.00..11613.33 rows=27 width=19)
        Filter: ((name)::text = '龙志'::text)
```

哈希索引组SQL语句执行计划如下

```
QUERY PLAN
Bitmap Heap Scan on students  (cost=4.50..242.10 rows=64 width=19)
  Recheck Cond: ((name)::text = '龙志'::text)
  ->  Bitmap Index Scan on students_name_idx  (cost=0.00..4.48 rows=64 width=0)
        Index Cond: ((name)::text = '龙志'::text)
```

B树索引组SQL语句执行计划如下

```
QUERY PLAN
Bitmap Heap Scan on students  (cost=4.92..242.53 rows=64 width=19)
  Recheck Cond: ((name)::text = '龙志'::text)
  ->  Bitmap Index Scan on students_name_idx  (cost=0.00..4.91 rows=64 width=0)
        Index Cond: ((name)::text = '龙志'::text)
```

我们可以看到，无索引组的查询生成计划采用了并行顺序查找的方式，其预估查询成本较高，而对于哈希索引与B树索引组，查询操作则借助了索引结构完成，其预估查询成本较低，同样的，实验过程中其查询速度也较快。同时分析实验数据我们发现，相较于单表100万数据组，单表1亿数据量下哈希索引的耗时存在明显增长，而B树索引的耗时不变，这可能是由于较大的数据量发生了更为频繁的哈希碰撞。

2. 分别对students表100万数据量、1亿数据量情况执行范围查询语句

首先将id字段的主键约束与索引取消，随后执行如下SQL语句

SQL语句5 SQL

```sql
select count(*) from students where id <100000 and id>89999;
```

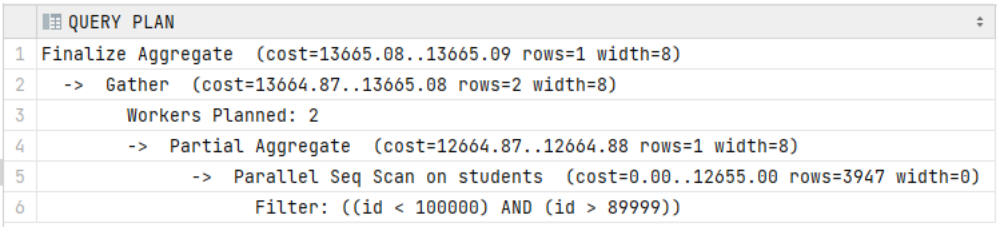
实验过程类似于3.1所述步骤，结果如下

不同系统与索引情况下范围查询语句耗时汇总

	单表100万数据	单表1亿数据
PostgreSQL 无索引	67ms	11782ms
PostgreSQL 哈希索引	70ms	12217ms
PostgreSQL B树索引	5ms	6ms
Java手动编程	300ms	/
MyDB CSV存储引擎	1002ms	/

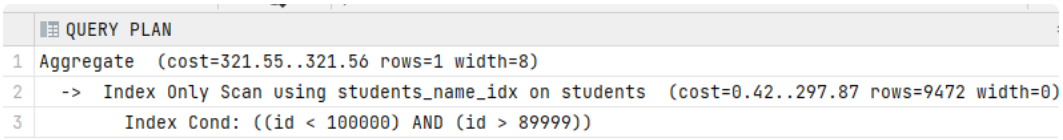
结果分析

同样执行explain语句查看哈希索引组的查询计划，其执行计划如下



```
QUERY PLAN
Finalize Aggregate  (cost=13665.08..13665.09 rows=1 width=8)
  ->  Gather  (cost=13664.87..13665.08 rows=2 width=8)
        Workers Planned: 2
        ->  Partial Aggregate  (cost=12664.87..12664.88 rows=1 width=8)
              ->  Parallel Seq Scan on students  (cost=0.00..12655.00 rows=3947 width=0)
                    Filter: ((id < 100000) AND (id > 89999))
```

B树索引组的执行计划如下



```
QUERY PLAN
Aggregate  (cost=321.55..321.56 rows=1 width=8)
  ->  Index Only Scan using students_name_idx on students  (cost=0.42..297.87 rows=9472 width=0)
        Index Cond: ((id < 100000) AND (id > 89999))
```

我们可以发现，对于范围查询语句，哈希索引组的执行计划采用了并行顺序遍历的方式，而B树索引仍能有效地借助索引结构完成查找，这契合了前文中哈希索引无法有效满足范围查询的分析。

3. 分别对students表100万数据量、1亿数据量情况执行模糊查询语句

SQL语句6–右模糊查询 SQL

```sql
select count(*) from students where name like '季%';
```

SQL语句7–全模糊查询 SQL

```sql
select count(*) from students where name like '%JCoder%';
```

实验过程类似于3.1所述步骤，结果如下

不同系统与索引情况下模糊查询语句耗时汇总

	右模糊查询		全模糊查询	
	单表100万数据	单表1亿数据	单表100万数据	单表1亿数据
PostgreSQL 无索引	83ms	12990ms	79ms	13011ms
PostgreSQL 哈希索引	80ms	13569ms	81ms	13380ms
PostgreSQL B树索引1	82ms	13161ms	83ms	13377ms
PostgreSQL B树索引2	3ms	64ms	80ms	14502ms
Java手动编程	283ms	/	284ms	/
MyDB CSV存储引擎	947ms	/	974ms	/

结果分析

对于B树索引组1，实验执行了如下SQL语句建立索引结构

```
create index students_name_idx on students using btree(name);
```

查看其执行SQL语句6的查询计划，查询计划如下

```
QUERY PLAN
Finalize Aggregate  (cost=12624.46..12624.47 rows=1 width=8)
  ->  Gather  (cost=12624.24..12624.45 rows=2 width=8)
        Workers Planned: 2
        ->  Partial Aggregate  (cost=11624.24..11624.25 rows=1 width=8)
              ->  Parallel Seq Scan on students  (cost=0.00..11613.33 rows=4363 width=0)
                    Filter: ((name)::text ~~ '季%'::text)
```

由上图我们发现，B树索引组1采用了顺序遍历的方式进行查询操作，并未借助索引结构，这是由于PostgreSQL在执行右模糊查询语句时需指定对字符串数据的排序方式，因此，B树索引组2执行了如下SQL语句重新建立索引结构

```
create index students_name_idx on students using btree(name collate "C");
```

查看组2执行SQL语句6的查询计划，查询计划如下

```
QUERY PLAN
Aggregate  (cost=297.79..297.80 rows=1 width=8)
  ->  Index Only Scan using students_name_idx on students  (cost=0.42..271.61 rows=10471 width=0)
        Index Cond: ((name >= '季'::text) AND (name < '孤'::text))
        Filter: ((name)::text ~~ '季%'::text)
```

我们可以发现，在指定排序顺序后，PostgreSQL数据库将右模糊查询转化为了范围查询，有效地借助索引结构完成查询操作，其查询速度非常快。

同时对于全模糊查询，其查询操作的执行计划如下

```
QUERY PLAN
Finalize Aggregate  (cost=12644.50..12644.51 rows=1 width=8)
  ->  Gather  (cost=12644.28..12644.49 rows=2 width=8)
        Workers Planned: 2
        ->  Partial Aggregate  (cost=11644.28..11644.29 rows=1 width=8)
              ->  Parallel Seq Scan on students  (cost=0.00..11613.33 rows=12380 width=0)
                    Filter: ((name)::text ~~ '%JCoder%'::text)
```

我们发现，全模糊查询语句无法简单地借助B树索引结构提高查询效率。

综合上述实验我们发现，**采用索引结构能够有效地提升数据库的查询语句执行效率**，而相较于哈希索引，B树索引能够契合多种查询语句需求，同时PostgreSQL系统对B树索引的实现也更为完善，因此，**对需执行高性能查询的字段建立B树索引效果更好**。同时在PostgreSQL系统中，右模糊查询语句需要在建立索引的过程中指定字符串的排序顺序。而对于全模糊查询语句等更加复杂的查询语句，B树结构可能无法满足这类查询语句的索引需要，因此，应考虑采用其它索引结构或在开发存在高性能查询需求时应尽量避免此类语句的存在，从而提升查询语句的执行效率。

数据更新操作

实验对不同方式下数据更新操作语句的执行进行比较

1. 在PostgreSQL控制台中执行如下语句

SQL语句3 SQL

```sql
update people set first_name = replace(first_name, 'To', 'TTOO'),
surname=replace(surname, 'To', 'TTOO')
where first_name like '%To%' or surname like '%To%';
```

执行结果如下：

304 rows | Tx: Auto | DDL

	peopleid	first_name	surname	born	died	gender
1	16431	Hans	van TTOOngeren	1955	1982	M
2	16443	TTOOmas	von Brömssen	1943	<null>	M
3	24	TTOOru	Abe	1917	1993	M
4	47	TTOOmmy	Abuel	1942	<null>	M
5	134	TTOOyin	Aimakhu	1984	<null>	F
6	212	TTOOmas	Alfredson	1965	<null>	M
7	264	TTOOm	Alter	1950	2017	M
8	419	TTOOmio	Aoki	1923	2004	M
9	494	TTOOdd	Armstrong	1937	1992	M
10	502	TTOOm	Arnold	1959	<null>	M
11	545	TTOOyoo	Ashida	1944	2011	M
12	586	TTOOm	Atkins	1935	<null>	M
13	587	TTOOmmy	Atkins	1887	1968	M
14	818	TTOOny	Bancroft	1967	<null>	M

2. 采用Java手动编程方式将表中first_name与surname字段中的To替换为TTOO，执行结果如下：

```
19,'Barkhad','Abdi',1985,NULL,'M'
20,'Kareem','Abdul-Jabbar',1947,NULL,'M'
21,'Aleksandr','Abdulov',1953,2008,'M'
22,'Hiroshi','Abe',1964,NULL,'M'
23,'Sadao','Abe',1970,NULL,'F'
24,'TTOOru','Abe',1917,1993,'M'
25,'Tsuyoshi','Abe',1982,NULL,'M'
26,'Alfred','Abel',1879,1937,'M'
27,'Jake','Abel',1987,NULL,'M'
28,'Raja','Abel',1978,NULL,'M'
29,'Lionel','Abelanski',1964,NULL,'M'
30,'Ian','Abercrombie',1934,2012,'M'
31,'Elke','Aberle',1950,NULL,'F'
32,'Ary','Abittan',1974,NULL,'M'
33,'Simon','Abkarian',1962,NULL,'M'
34,'Alon','Aboutboul',1965,NULL,'M'
```

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\j
总计耗时：92毫秒

Process finished with exit code 0
```

3. 采用MyDB模拟系统执行 SQL语句3，执行结果如下：

```
"19","Barkhau","Abdi","1985","null","M"
"20","Kareem","Abdul-Jabbar","1947","null","M"
"21","Aleksandr","Abdulov","1953","2008","M"
"22","Hiroshi","Abe","1964","null","M"
"23","Sadao","Abe","1970","null","F"
"24","TTOOru","Abe","1917","1993","M"
"25","Tsuyoshi","Abe","1982","null","M"
"26","Alfred","Abel","1879","1937","M"
"27","Jake","Abel","1987","null","M"
"28","Raja","Abel","1978","null","M"
"29","Lionel","Abelanski","1964","null","M"
"30","Ian","Abercrombie","1934","2012","M"
"31","Elke","Aberle","1950","null","F"
"32","Ary","Abittan","1974","null","M"
"33","Simon","Abkarian","1962","null","M"
"34","Alon","Aboutboul","1965","null","M"
"35","F. Murray","Abraham","1939","null","M"
"36","John","Abraham","1972","null","M"
"37","Jim","Abrahams","1944","null","M"
"38","Jon","Abrahams","1977","null","M"
"39","Lenny","Abrahamson","1966","null","M"
"40","Cláudia","Abreu","1970","null","F"
```

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" ...
Welcome to MyDB! Input SQL Command to control the system.
update people set first_name = replace(first_name, 'To', 'TTOO'), surname=replace(surname, 'To', 'TTOO') where first_name like '%To%' or surname like '%To%';
Executing sql command: update people set first_name = replace(first_name, 'To', 'TTOO'), surname=replace(surname, 'To', 'TTOO') where first_name like '%To%' or surname like '%To%';
Rows affected: 304
耗时：232毫秒
```

系统	PostgreSQL 15	Java 手动编程	MyDB系统
耗时	0.028s	0.092s	0.232s

我们可以看到，PostgreSQL系统执行数据更新操作的效率略高于采用Java手动编程等方式的执行效率，这是受前文中所分析的数据库系统实现等因素决定的。对于PostgreSQL系统，当对行数据进行更新操作时，若采用全表遍历操作且更新后行占用字节小于等于原先占用字节，则会采取直接修改的操作完成更新，反之则采用删除–插入的操作实现对数据的更新。

数据一致性探究

在实际开发与运行过程中，保证数据库内部数据的一致性至关重要。数据一致性强调数据库运行过程中数据始终处于正确的状态，不出现数据间的不兼容或不匹配现象[3]。但在实际开发与运营过程中，由于数据库本身或物理运行设备存在故障可能，因此可能会发生数据操作中断，从而对数据的一致性造成威胁。同时，出于支持并发访问的需要，数据库系统可能会同时接收不同连接所发送的数据操作集合，若不进行处理，可能会出现数据操作顺序混乱，从而破坏数据的一致性。其次，若只对单表内部进行表内约束，则在跨表操作执行的过程中，可能会出现因数据操作失误造成的表间信息不对应，从而造成表间数据的不一致。因此，数据库系统需采取一系列措施，以保障数据库的数据一致性。

PostgreSQL数据库系统主要采用事务模型与外键模型保障数据一致性，数据库系统通过对这两种模型的完整实现，有效保障了数据的一致性。

事务模型

事务模型是数据库系统中的重要概念，通过将不可分割的一系列的数据操作打包为一组事务，并指定该事务仅存在两种运行结果：运行失败，全部操作失效；运行成功，全部操作生效，该模型能够保证数据库系统不会因数据库系统或物理运行设备故障而出现仅存在部分操作生效的情况，从而有效保障了数据的一致性。

PostgreSQL系统采用WAL日志技术实现对事务模型的完整支持，WAL日志技术通过在对数据或表文件执行操作前先将该操作以特定形式记录至日志文件种，从而实现了数据操作的可回溯性。当因系统故障等原因发生事务夭折时，数据库系统将依照WAL日志逐步撤销已执行的操作，从而实现将数据回滚至事务开始前的数据状态，保障了数据的一致性。

为对事务模型的有效性进行探究，实验模拟转账操作设计了bank.sql数据表与初始数据，并分别采用手动编程文件操作（无事务模型）与PostgreSQL数据库系统数据操作（开启事务模型）两种方式模拟两账户间进行转账，探究系统故障操作意外中断对数据表数据的影响，为便于模拟实验的开展，两组实验均模拟一次usr2向usr1进行一次10k元的转账，每次转账均以1元为单位逐次扣除usr2的存款，再以1元为单位逐次增加usr1的存款，该实验的具体过程如下。

手动编程文件操作（无事务模型）方式下系统故障对数据的影响实验

1. 采用Java手动编程的方式导入bank.sql的表结构与数据内容，并实现CSVDatabase以实现转账操作，导入后的结果如下，在初始数据中，usr2与usr1各有10000元的存款

```
"C:\Program Files\Java\jdk-17.0.4.1\
Table:account
name        deposit
usr1        10000
usr2        10000

Process finished with exit code 0
```

2. 模拟一次转账操作，在TransactionTest类中将Database设置为CSVDatabase，运行bankTransfer执行转账操作，待转账操作完成后，再运行selectAccount函数查看运行结果，运行结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin
Table:account
name        deposit
usr1        20000
usr2        0

Process finished with exit code 0
```

可以看到，转账操作成功，usr2成功向usr1转账了10k元

3. 执行reset函数恢复表的初始数据，再次执行转账操作，在转账运行结束前访问该数据表，此时程序

会因进程无法访问account数据表而抛出运行异常，从而模拟了转账操作的意外终止

```
processing... remain:9386
processing... remain:9385
processing... remain:9384

java.lang.RuntimeException: java.nio.file.FileSystemException: Data\account.csv: 另一个程序正在使用此文件，进程无法访问。

    at experiment.CSVDB.CSVDatabase.update(CSVDatabase.java:19)
    at experiment.CSVDB.CSVDatabase.increaseAmount(CSVDatabase.java:31)
    at experiment.CSVDB.CSVDatabase.increaseAccountAmount(CSVDatabase.java:60)
    at transaction.TransactionTest.bankTransfer(TransactionTest.java:11) <27 internal lines>
Caused by: java.nio.file.FileSystemException Create breakpoint : Data\account.csv: 另一个程序正在使用此文件，进程无法访问。
    at java.base/sun.nio.fs.WindowsException.translateToIOException(WindowsException.java:92)
    at java.base/sun.nio.fs.WindowsException.rethrowAsIOException(WindowsException.java:103)
    at java.base/sun.nio.fs.WindowsException.rethrowAsIOException(WindowsException.java:108)
    at java.base/sun.nio.fs.WindowsFileSystemProvider.implDelete(WindowsFileSystemProvider.java:275)
    at java.base/sun.nio.fs.AbstractFileSystemProvider.delete(AbstractFileSystemProvider.java:105)
    at java.base/java.nio.file.Files.delete(Files.java:1152)
    at experiment.CSVDB.CSVHandler.drop(CSVHandler.java:30)
    at experiment.CSVDB.CSVDatabase.update(CSVDatabase.java:16)
    ... 30 more


Process finished with exit code -1
```

再运行select函数查看该表数据，查询结果如下

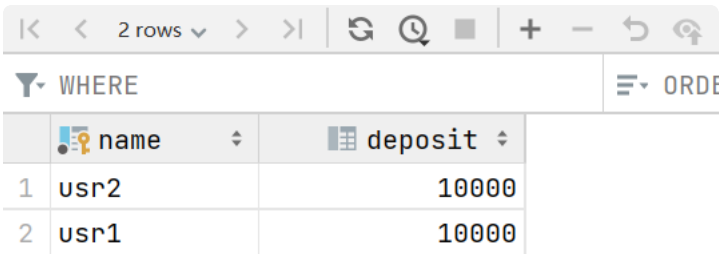
```
"C:\Program Files\Java\jdk-17.0.4.1\
Table:account
name        deposit
usr1        10000
usr2        9384

Process finished with exit code 0
```

此时，数据表数据进入了usr2部分扣款而usr1未收到转账的状态，数据表总金额发生了改变，出现了数据的不一致情况。

数据库系统开启事务模型方式下系统故障对数据的影响实验

1. 在Postgresql数据库中导入bank.sql文件，并采用JDBC的方式连接PostgreSQL数据库，采用事务模型实现PostgreSQLDatabase的转账功能。bank.sql文件导入结果如下，可以看到数据已被成功导入



2. 运行bankTransfer函数模拟执行一次转账操作，运行完成后查看account表数据，运行结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin
Table:account(PostgreSQL)
name        deposit
----------------------------
usr1        20000
usr2        0

Process finished with exit code 0
```

可以看到转账操作可以被成功执行

3. 运行reset函数恢复表数据，再次执行转账操作，在转账运行结束前中止程序运行，模拟转账操作的意外终止

```
processing... remain:915
processing... remain:914
processing... remain:913
processing... remain:912
processing... remain:911
processing... remain:910
processing... remain:909
processing... remain:908
processing... remain:907
processing... remain:906
processing... remain:905

Process finished with exit code 130
```

随后运行selectAccount函数查看该表数据，结果如下

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\
Table:account(PostgreSQL)
name        deposit
----------------------------
usr1        10000
usr2        10000

Process finished with exit code 0
```

可以看到，在事务发生了意外终止后，数据表中的数据并未更新，这符合事务夭折后，事务内全部操作不生效的设计。

通过对比无事务模型与有事务模型下数据库系列操作意外终止实验的实验结果，我们发现，开启事务模型能够有效地保证系列数据操作的原子性，避免系列操作仅有部分成功执行而带来的数据不一致问题。因此，**事务模型能够有效地保障数据库的数据一致性**。

外键模型

外键模型是数据库系统中的重要模型，它通过设定表中某一字段的值指向另一张表的主键，从而实现在两个表之间的数据关联，在执行数据操作时，数据库系统会检查表间外键与主键的联系，保证外键不会指向不存在的主键数据，从而一定程度上避免了表间数据的不匹配，保障了表间数据的一致性。PostgreSQL系统实现了对表间外键的支持，同时提供了对出现外键冲突时的多种处理方法。实验采用orders.sql文件对PostgreSQL数据库系统中的外键模型的有效性进行测试。

外键约束与冲突处理实验

1. 实验以order.sql文件作为测试数据，对表间外键的有效性进行测试，运行如下测试代码

```sql
--为test用户创建一条订单信息
insert into orders(order_id, customer_id) VALUES (2,3);
insert into order_items(order_id, item_id, item_amount) VALUES (2,1,1);
--外键约束违反测试
insert into order_items(order_id, item_id, item_amount) VALUES (2,4,1);--item_id不存在
update order_items set order_id=3 where order_id=2;--order_id不存在
delete from orders where order_id=2;--删除订单被order_items引用
```

测试结果如下

```
 insert into orders(order_id, customer_id) VALUES (2,3)
1:09:12] 1 row affected in 4 ms
 insert into order_items(order_id, item_id, item_amount) VALUES (2,1,1)
1:09:16] 1 row affected in 4 ms
 insert into order_items(order_id, item_id, item_amount) VALUES (2,4,1)
1:09:19] [23503] 错误: 插入或更新表 "order_items" 违反外键约束 "order_items_item_id_fkey"
1:09:19] 详细：键值对(item_id)=(4)没有在表"items"中出现.
 update order_items set order_id=3 where order_id=2
1:09:21] [23503] 错误: 插入或更新表 "order_items" 违反外键约束 "order_items_order_id_fkey"
1:09:21] 详细：键值对(order_id)=(3)没有在表"orders"中出现.
 delete from orders where order_id=2
1:09:24] [23503] 错误: 在 "orders" 上的更新或删除操作违反了在 "order_items" 上的外键约束 "order_items_order_id_fkey"
1:09:24] 详细：键值对(order_id)=(2)仍然是从表"order_items"引用的.
```

可以看到，对于违反表间外键关系的操作（插入/更新操作引用不存在的主键、删除行主键被外部表所依赖），数据库系统均拒绝了执行，体现数据库外键能够有效地保证数据库间的数据一致性

2. 实验采用同样数据对自表外键进行测试，运行如下代码

自表外键测试 SQL

```sql
--插入一条用户并将parent引用至主键为1的customer
insert into customers(id, name, parent_id) VALUES (4,'test2',1);
--自表外键约束违反测试
insert into customers(id, name, parent_id) VALUES (5,'test3',6);
delete from customers where id=1;--删除用户被引用
```

测试结果如下

```
:> insert into customers(id, name, parent_id) VALUES (4,'test2',1)
21:19:08] 1 row affected in 2 ms
:> insert into customers(id, name, parent_id) VALUES (5,'test3',6)
21:19:22] [23503] 错误: 插入或更新表 "customers" 违反外键约束 "customers_parent_id_fkey"
21:19:22] 详细: 键值对(parent_id)=(6)没有在表"customers"中出现.
:> delete from customers where id=1
21:19:27] [23503] 错误: 在 "customers" 上的更新或删除操作违反了在 "customers" 上的外键约束 "customers_parent_id_fkey"
21:19:27] 详细: 键值对(id)=(1)仍然是从表"customers"引用的.
```

可以看到，数据库同样拦截了违反自表外键引用的操作，保证了表内行数据关系的一致性。

3. 在上述实验中，对于违反外键关系的数据删除操作，PostgreSQL数据库系统对该操作采取了拒绝执行的策略。除拒绝执行外，postgresql数据库系统还支持对该类删除操作采取级联删除与设空策略，实验对此进行探究。

   a. 将order_items的order_id外键约束修改为级联删除，执行如下语句

设置级联删除 SQL

```sql
alter table order_items drop constraint order_items_order_id_fkey;
alter table order_items add constraint order_items_order_id_fkey
    foreign key (order_id) references orders(order_id) on delete cascade ;
```

随后执行对order表中数据的删除操作，代码如下

```sql
delete from orders where order_id=2;
```

随后查看order_items表

order_id	item_id	item_amount
1	1	10
1	2	1
1	3	5

可以看到，先前所插入的order_id=2的行数据也一并被删除

b. 将customers的parent字段的外键约束修改为删除设空，执行如下语句

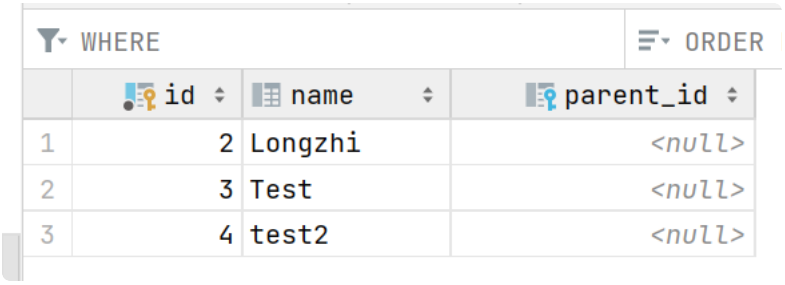
设置删除设空 SQL

```sql
alter table customers drop constraint customers_parent_id_fkey;
alter table customers add constraint customers_parent_id_fkey
    foreign key (parent_id) references customers(id) on delete set null ;
--同时删除其它的表对customers的主键依赖
```

随后对customers表中数据执行删除操作，执行如下代码

```sql
delete from customers where id=1;
```

查看customers表，可以看到test2原先对id=1数据的引用自动修改为了null



WHERE ORDER

	id	name	parent_id
1	2	Longzhi	<null>
2	3	Test	<null>
3	4	test2	<null>

通过上述实验，我们可以看到，PostgreSQL系统完整地实现了表内外键约束与表间外键约束的支持，同时提供了多种方法处理在数据操作过程中发生外键冲突时所采取的应对策略，从而更有效地保障了数据的一致性。

导入操作外键约束实验

由于外键约束对数据间的引用关系存在严格约束，因此限制了同事务中数据间的操作顺序，可能会对执行数据库恢复导入等操作产生影响，实验对此进行探究，操作过程如下。

1. 调换order.sql中insert语句的顺序，将order_items表插入数据语句提前，修改后结果如下

```
insert into order_items (order_id, item_id,item_amount) values (1, 1, 10);
insert into order_items (order_id, item_id,item_amount) values (1, 2, 1);
insert into order_items (order_id, item_id,item_amount) values (1, 3, 5);
insert into customers (id, name) values (1, 'Fei');
insert into customers (id, name) values (2, 'Longzhi');
insert into customers (id, name) values (3, 'Test');
insert into items (name, price) values ('PostgreSQL', 10);
insert into items (name, price) values ('MySQL', 20);
insert into items (name, price) values ('Oracle', 30);
insert into orders (customer_id) values (1);
commit;
```

2. 运行该事务，我们可以看到，尽管若该事务全部执行完成不会出现数据的不一致，但因为事务执行至对order_items表的插入操作时，数据库抛出了操作错误的提示

```
22  insert into order_items (order_id, item_id,item_amount) values (1, 1, 10);
23  insert into order_items (order_id, item_id,item_amount) values (1, 2, 1);
24  insert into order_items (order_id, item_id,item_amount) values (1, 3, 5);
```

order_items ›

[23503] 错误: 插入或更新表 "order_items" 违反外键约束 "order_items_order_id_fkey"　Stop　Retry　Ignore　Ignore All
详细：键值对(order_id)=(1)没有在表"orders"中出现.

3. 开启外键约束延迟检查，将数据表外键属性修改为 deferrable initially deferred，如下图所示，在此属性下，数据库将会将外键检查延迟至事务结束以后

```
create table customers(
    id int primary key ,
    name varchar,
    parent_id int default null references customers(id) deferrable initially deferred
);
create table items (
    item_id serial primary key,
    name varchar,
    price int check ( price > 0 )
 );
create table orders (
        order_id serial primary key,
        customer_id int references customers(id) deferrable initially deferred
);
create table order_items (
        order_id int references orders(order_id) deferrable initially deferred,
        item_id int references items(item_id) deferrable initially deferred,
        item_amount int check ( item_amount > 0 ),
        primary key (order_id, item_id)
);
```

再次运行该事务，结果如下，该事务被成功运行，并未出现冲突

```
00:44:36] completed in 6 ms
> create table orders (
          order_id serial primary key,
          customer_id int references customers(id) deferrable initially deferred
  )
00:44:36] completed in 5 ms
> create table order_items (
          order_id int references orders(order_id) deferrable initially deferred,
          item_id int references items(item_id) deferrable initially deferred,
          item_amount int check ( item_amount > 0 ),
          primary key (order_id, item_id)
  )
00:44:36] completed in 3 ms
> insert into order_items (order_id, item_id,item_amount) values (1, 1, 10)
00:44:36] 1 row affected in 1 ms
> insert into order_items (order_id, item_id,item_amount) values (1, 2, 1)
00:44:36] 1 row affected in 2 ms
```

总结

外键模型是对数据库系统数据一致性的强力约束，它能够有效地保障行数据字段不会引用不存在的行数据，从而保证了在数据分表等情形下的数据一致性。在上述实验中我们发现，**PostgreSQL数据库系统完整地实现了对外键模型的支持，能够有效阻止违反外键约束的非法数据操作，也实现了对违反外键约束操作的多种响应策略，同时支持延迟外键检查等多种功能**。但在现代开发环境下，违背外键约束的操作往往是由于开发过程的失误而导致的，在保障开发代码质量与数据库操作规范的条件下，数据操作往往不会违背外键约束。同时，数据库系统在实现外键约束检查过程中可能会执行一定的查询操作，在表数据量大与分布式系统种，这些操作可能会占用大量的资源。因**此，在现代大型应用程序中，往往不采用外键约束模型，而是通过加强代码质量与维护过程的规范性，从而保证数据的一致性**。

数据完整性探究

数据完整性强调数据库中所存数据的正确性。在关系型数据库系统中，数据完整性往往包含三类约束，第一类为实例完整性约束，要求保证关系中元组的唯一可识别性，即主键唯一性，第二类约束是参照完整性约束，即跨表之间行数据之前存在正确的引用关系，即外键约束，而第三类约束为用户定义的完整性约束，通常为用户所自定义的非空约束、范围约束等约束，用以保证数据库数据符合现实世界。

现有数据库系统可以有效保障数据完整性，如PostgreSQL系统支持多种约束，如非空约束、唯一约束、主键约束、外键约束等约束限制，同时，PostgreSQL系统支持对支持对字段所存储数据执行类型检查，从而保证表结构与数据的完整性，实验对此进行探究。

单表完整性检查

非法插入测试

在非法插入测试中，实验执行了SQL创建语句1创建测试用表，随后执行SQL测试语句1对表中的主键约束、唯一约束、非空约束、数据类型约束、检查约束、外键约束进行了测试

SQL创建语句1 SQL

```sql
create table insert_test(
    id int primary key ,
    name varchar(8) unique ,
    age int check ( age>=0 and age<=120 ),
    birthday date not null ,
    parent int references insert_test(id)
);
```

SQL测试语句1 SQL

```sql
insert into insert_test(id, name, age, birthday, parent)
values(1,'数据库',24,'2023-10-15',null);--执行成功
insert into insert_test(id,name,age,birthday,parent)
values (1,'数据库2',24,'2023-10-15',null);--违反主键约束
insert into insert_test(id,name,age,birthday,parent)
values (2,'数据库',24,'2023-10-15',null);--违反唯一约束
insert into insert_test(id,name,age,birthday,parent)
values (2,'数据库2',-114,'2023-10-15',null);--违反检查约束
insert into insert_test(id,name,age,birthday,parent)
values (2,'数据库2',24,'2023-2-29',null);--违反日期约束
insert into insert_test(id,name,age,birthday,parent)
values (2,'数据库2',24,2023,null);--违反数据类型
insert into insert_test(id,name,age,birthday,parent)
values (2,'数据库2',24,'2023-10-15',3);--违反外键约束
insert into insert_test(id,name,age,birthday,parent)
values (2,'123456789',24,'2023-10-15',null);--违反数据大小限制
insert into insert_test(id,name,age,birthday,parent)
values (2147483648,'数据库2',24,'2023-10-15',null);--int值溢出
insert into insert_test(id,name,age,birthday,parent)
values (2,'数据库2',24,null,null);--违反非空约束
```

实验结果如下

```
HugeData.public> insert into insert_test(id, name, age, birthday, parent) values(1,'数据库',24,'2023-10-15',null)
[2023-10-07 20:15:48] 1 row affected in 3 ms
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (1,'数据库2',24,'2023-10-15',null)
[2023-10-07 20:18:44] [23505] 错误: 重复键违反唯一约束"insert_test_pkey"
[2023-10-07 20:18:44] 详细: 键值"(id)=(1)" 已经存在
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2,'数据库',24,'2023-10-15',null)
[2023-10-07 20:18:52] [23505] 错误: 重复键违反唯一约束"insert_test_name_key"
[2023-10-07 20:18:52] 详细: 键值"(name)=(数据库)" 已经存在
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2,'数据库2',-114,'2023-10-15',null)
[2023-10-07 20:18:57] [23514] 错误: 关系 "insert_test" 的新列违反了检查约束 "insert_test_age_check"
[2023-10-07 20:18:57] 详细: 失败, 行包含(2, 数据库2, -114, 2023-10-15, null).
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2,'数据库2',24,'2023-2-29',null)
[2023-10-07 20:19:01] [22008] 错误: 日期/时间值超出范围: "2023-2-29"
[2023-10-07 20:19:01] 位置: 74
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2,'数据库2',24,2023,null)
[2023-10-07 20:19:06] [42804] 错误: 字段 "birthday" 的类型为 date, 但表达式的类型为 integer
[2023-10-07 20:19:06] 建议: 你需要重写或转换表达式
[2023-10-07 20:19:06] 位置: 74
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2,'数据库2',24,'2023-10-15',3)
[2023-10-07 20:19:12] [23503] 错误: 插入或更新表 "insert_test" 违反外键约束 "insert_test_parent_fkey"
[2023-10-07 20:19:12] 详细: 键值对(parent)=(3)没有在表"insert_test"中出现.
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2,'123456789',24,'2023-10-15',null)
[2023-10-07 20:31:57] [22001] 错误: 对于可变字符类型来说，值太长了(8)
HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2147483648,'数据库2',24,'2023-10-15',null)
[2023-10-07 20:32:06] [22003] 错误: 整数超出范围

HugeData.public> insert into insert_test(id,name,age,birthday,parent) values (2,'数据库2',24,null,null)
[2023-10-07 20:38:42] [23502] 错误: null value in column "birthday" of relation "insert_test" violates not-null constraint
[2023-10-07 20:38:42] 详细: 失败, 行包含(2, 数据库2, 24, null, null).
```

可以看到，PostgreSQL系统有效拦截了所有非法插入语句，尤其是对较为特殊的非法日期插入与超范围整数插入均能有效过滤。

非法修改测试

执行SQL测试语句2，对已有两条数据的数据表修改表字段参数、检查是否能执行违背约束与数据类型限制的操作

SQL测试语句2 SQL

```sql
insert into insert_test(id,name,age,birthday,parent) values (2,null,24,'2023-10-15',null);
--现有数据表中有id=1、2两条数据
alter table insert_test alter column name type text;--执行成功
alter table insert_test alter column id type int2;--执行成功
alter table insert_test alter column name type varchar(1);--与表已有数据大小冲突
alter table insert_test alter column name type date;--与表已有数据类型冲突
alter table insert_test alter column name set not null ;--非空约束与已有数据冲突
alter table insert_test
add constraint unique_name unique(birthday);--唯一约束与已有数据冲突
alter table insert_test
add constraint check_age check ( age<0 );--检查约束与已有数据冲突
alter table insert_test
add constraint pk2 primary key (name);--不可指定多个主键
alter table insert_test
drop constraint insert_test_pkey;--存在外键依赖该主键
alter table insert_test
drop constraint insert_test_pkey cascade ,
add constraint pk2 primary key (name);--已有数据违反主键约束
```

执行结果如下

```
HugeData.public> alter table insert_test alter column name type text
[2023-10-07 20:53:49] completed in 8 ms
HugeData.public> alter table insert_test alter column id type int2
[2023-10-07 20:53:52] completed in 10 ms
HugeData.public> alter table insert_test alter column name type varchar(1)
[2023-10-07 20:53:56] [22001] 错误: 对于可变字符类型来说，值太长了(1)
HugeData.public> alter table insert_test alter column name type date
[2023-10-07 20:53:58] [42804] 错误: 字段 "name" 不能自动转换成类型 date
[2023-10-07 20:53:58] 建议: 您可能需要指定"USING name::date"。
HugeData.public> alter table insert_test alter column name set not null
[2023-10-07 20:54:02] [23502] 错误: column "name" of relation "insert_test" contains null values
HugeData.public> alter table insert_test add constraint unique_name unique(birthday)
[2023-10-07 20:54:09] [23505] 错误: 无法创建唯一索引"unique_name"
[2023-10-07 20:54:09] 详细: 键值(birthday)=(2023-10-15)重复了
HugeData.public> alter table insert_test add constraint check_age check ( age<0 )
[2023-10-07 20:54:16] [23514] 错误: check constraint "check_age" of relation "insert_test" is violated by some row
HugeData.public> alter table insert_test add constraint pk2 primary key (name)
[2023-10-07 20:54:19] [42P16] 错误: 对表 "insert_test" 指定多个主键是不允许的
HugeData.public> alter table insert_test drop constraint insert_test_pkey ,add constraint pk2 primary key (name)
[2023-10-07 20:55:11] [2BP01] 错误: 无法删除 在表 insert_test上的约束insert_test_pkey  因为有其它对象倚赖它
[2023-10-07 20:55:11] 详细: 在表 insert_test上的约束insert_test_parent_fkey  倚赖于 索引 insert_test_pkey
[2023-10-07 20:55:11] 建议: 使用 DROP .. CASCADE 把倚赖对象一并删除.
HugeData.public> alter table insert_test drop constraint insert_test_pkey cascade ,add constraint pk2 primary key (name)
[2023-10-07 20:56:22] [23502] 错误: column "name" of relation "insert_test" contains null values
```

可以看到，PostgreSQL同样有效拦截了所有对数据表属性参数的非法修改。//TODO并且，PostgreSQL系统为每个非法修改都提供了报错

非法删除测试

非法删除操作主要集中于外键约束中，实验执行SQL测试语句3，对已有数据表进行数据删除与约束删除操作，以检查是否能执行违背外键约束的删除操作

SQL测试语句3 SQL

```sql
insert into insert_test(id,name,age,birthday,parent) values (3,'数据库3',24,'2023-10-15',1);
--现在数据库中有三条数据，id=1、2、3
delete from insert_test where id=2;--执行成功
delete from insert_test where id=1;--存在外键依赖该主键
alter table insert_test drop constraint insert_test_pkey;--存在外键依赖该主键
delete from insert_test where birthday='2023-10-15';--执行成功
```

执行结果如下

```
HugeData.public> delete from insert_test where id=2
[2023-10-07 21:05:20] 1 row affected in 2 ms
HugeData.public> delete from insert_test where id=1
[2023-10-07 21:05:24] [23503] 错误: 在 "insert_test" 上的更新或删除操作违反了在 "insert_test" 上的外键约束 "insert_test_parent_fkey"
[2023-10-07 21:05:24] 详细: 键值对(id)=(1)仍然是从表"insert_test"引用的.
HugeData.public> alter table insert_test drop constraint insert_test_pkey
[2023-10-07 21:05:29] [2BP01] 错误: 无法删除 在表 insert_test上的约束insert_test_pkey 因为有其它对象倚赖它
[2023-10-07 21:05:29] 详细: 在表 insert_test上的约束insert_test_parent_fkey 倚赖于 索引 insert_test_pkey
[2023-10-07 21:05:29] 建议: 使用 DROP .. CASCADE 把倚赖对象一并删除.
HugeData.public> delete from insert_test where birthday='2023-10-15'
[2023-10-07 21:05:32] 2 rows affected in 5 ms
```

可以看到，PostgreSQL系统同样拦截了所有的非法删除操作。

总结

综合以上三组实验，我们可以看到，**PostgreSQL系统对表约束的处理操作十分严格，能够有效拦截所有测试中违反约束的操作**。事实上，在官方文档中[4]，PostgreSQL定义了一系列可能违背表完整性的非法操作。PostgreSQL系统同时实现了对着一系列非法操作的拦截并经过了长期的测试，因此具备较高的质量保证。实验表明，**在良好的表约束定义下，PostgreSQL能够有效地保障数据表的完整性，具有较高的数据安全性，也在一定程度上减少了应用开发过程中所需要进行的数据过滤操作**。

表损坏检查与修复

在实际的生产与线上运行中，数据表可能因多种原因出现损坏情况，从而对数据的持久性造成威胁。因此，对数据表进行损坏检查与损坏数据修复工作至关重要。PostgreSQL官方文档中提供了三种备份方式[4]，分别为物理备份，逻辑备份与PRIP备份，同时提供了块数据校验技术以检测损坏数据的存在，有效保障了数据的一致性与持久性。

物理备份

物理备份是最基础的数据库备份形式，即对数据库的实际存储文件进行复制操作，并将复制文件转移至外部存储设备中。采用Java手动管理数据文件读写的方式仅支持物理备份，在Windows系统下，可以使用类似于如下指令进行操作。

指令示例 Shell

```
copy students.csv D:\data\
```

运行如上指令，我们可以在指定的文件夹中看到所备份的csv文件。



同样，该操作也可以应用于PostgreSQL系统中，通过查看PostgreSQL系统所指定的数据库文件地址，可以采用类似的方法进行复制备份，再通过覆盖的方式恢复文件数据，从而实现对数据的物理备份与恢复。

然而，无论是对于基于手动编程文件读写还是采用PostgreSQL系统实现数据管理，这种基于文件系统进行手动备份的操作对于数据文件复杂的情况下均较为繁琐，容易出现因手动复制操作错误造成备份数据的部分或全部损失，同时备份期间需暂停数据库系统的全部服务，亦不支持指定数据库的部分表数据进行备份。尽管可采用一致性快照等方式避免备份期间暂停数据库系统的服务，一致性备份仍无法上述解决全部问题。**在PostgreSQL系统中，提供了除文件系统级物理备份外，更为高效与可靠的备份方式。**

逻辑备份

除物理备份外，PostgreSQL系统提供了逻辑备份方式以提高数据备份的效率与可靠性。逻辑备份使用pg_dump指令，需定期将数据库的结构与数据导出为SQL文件，在发生故障时再使用pg_restore指令将备份的SQL文件恢复至数据库中。

PostgreSQL所提供的逻辑备份具有易于操作，无需停止数据库运行即可进行备份与恢复操作，备份文件跨版本兼容PostgreSQL系统的特点[4]。但同时，逻辑备份存在数据库大时导入与导出效率低、只能实现数据部分恢复，无法修复损坏数据的缺陷。因此，实验对逻辑备份的相关特性进行了探究

1. 基本逻辑备份操作

执行如下操作，创建数据表并导入数据

初始数据创建指令 SQL

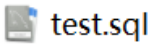
```sql
create database test;
create table students(
    id serial primary key ,
    studentid varchar(10),
    name varchar
);
copy students from '1k.csv' delimiter ',';
```

执行备份操作

备份指令 Shell

```shell
pg_dump -d Test -f test.sql -U postgres -w
//pg_dump -d Test114514 -f test.sql -U postgres -w
```

可以在文件夹中看到所备份的test.sql文件



现在删除原数据表，执行恢复指令

恢复指令 Shell

```shell
psql -d Test -U postgres -f test.sql
```

可以在数据库中看到原表已被恢复

WHERE ORDER BY

	id	studentid	name
501	501	501	章悦
502	502	502	李恩来
503	503	503	邓志丈
504	504	504	崔民
505	505	505	杜旺
506	506	506	朴泽
507	507	507	和亦菲
508	508	508	马亦菲
509	509	509	昌慧嘉
510	510	510	杜嘉乐
511	511	511	顾涵涵
512	512	512	余东东
513	513	513	黄滢滢
514	514	514	严大帝
515	515	515	严淑华
516	516	516	郎正熙
517	517	517	周欣源
518	518	518	昌健发

我们可以看到，采用逻辑备份可以较为方便地备份数据，当发生数据丢失时，可以将表恢复至备份时间点状态。但在逻辑备份时间点至故障发生时间点期间，如对表状态进行了修改操作，这部分修改操作将无法被恢复。因此，如采用逻辑备份进行备份，仍需承担部分数据丢失的风险，同时，逻辑备份也需要定期进行指令操作。

2. 不同数据量下逻辑备份与恢复的速度测试

为探究逻辑备份在不同数据量下的导入与导出效率，实验在数据库中创建10张表格，分别截取huge.csv前一千、前一万、前10万、前100万、前1000万五组不同量级的数据作为单表数据，进行数据备份效率测试。同时，由于PostgreSQL系统支持采用并行转储加快大数据量的转储速度，实验设置了不开启并行转储、并行转出参数分别为2、4、8三组实验进行测试。

本测试使用Windows Powershell控制台进行指令输入，使用Measure-Command关键字计算指令耗时

导入操作测试指令如下

测试指令 Shell

```
Measure-Command{pg_dump -d Test -f test.sql -U postgres}
Measure-Command{pg_dump -j 2 -Fd -f test -U postgres Test}
Measure-Command{pg_dump -j 4 -Fd -f test -U postgres Test}
Measure-Command{pg_dump -j 8 -Fd -f test -U postgres Test}
```

测试结果如下

单表	1k	10k	100k	1M	10M
不并行转储	0.139s	0.244s	0.346s	3.341s	50.582s
并行转储并行度2	0.220s	0.257s	0.606s	4.413s	40.757s
并行转储并行度4	0.337s	0.272s	0.464s	2.705s	25.988s
并行转储并行度8	0.361s	0.371s	0.473s	2.038s	17.951s

采用同样方法，实验测试了仅在数据库中只有一张表的情况下，单表一万、10万、100万、1000万、一亿数据量的逻辑备份操作，实验结果如下

单表	10k	100k	1M	10M	100M
不并行转储	0.123s	0.249s	0.365s	3.282s	51.384s
并行转储并行度2	0.218s	0.303s	0.919s	7.516s	65.177s
并行转储并行度4	0.297s	0.365s	0.904s	7.937s	65.485s
并行转储并行度8	0.409s	0.383s	0.924s	7.875s	65.575s

对上述实验中的转储文件进行恢复操作，操作指令如下

测试指令 Shell

```
Measure-Command{pg_dump -d Test -f test.sql -U postgres}
Measure-Command{pg_dump -j 2 -Fd -f test -U postgres Test}
Measure-Command{pg_dump -j 4 -Fd -f test -U postgres Test}
Measure-Command{pg_dump -j 8 -Fd -f test -U postgres Test}
 //


Measure-Command{pg_dump -d Test114514 -f test.sql -U postgres}
Measure-Command{pg_dump -j 2 -Fd -f test -U postgres Test114514}
Measure-Command{pg_dump -j 4 -Fd -f test -U postgres Test114514}
Measure-Command{pg_dump -j 8 -Fd -f test -U postgres Test114514}
```

恢复指令 Shell

```
Measure-Command{psql -d Test -U postgres -f test.sql}
Measure-Command{pg_restore -d Test  -j 2 -U postgres test}
Measure-Command{pg_restore -d Test  -j 4 -U postgres test}
Measure-Command{pg_restore -d Test  -j 8 -U postgres test}
 //
Measure-Command{psql -d Test6 -U postgres -f test.sql}
Measure-Command{pg_restore -d Test6  -j 2 -U postgres test}
Measure-Command{pg_restore -d Test6  -j 4 -U postgres test}
Measure-Command{pg_restore -d Test6  -j 8 -U postgres test}
```

数据库有10张表时数据导入速度测试

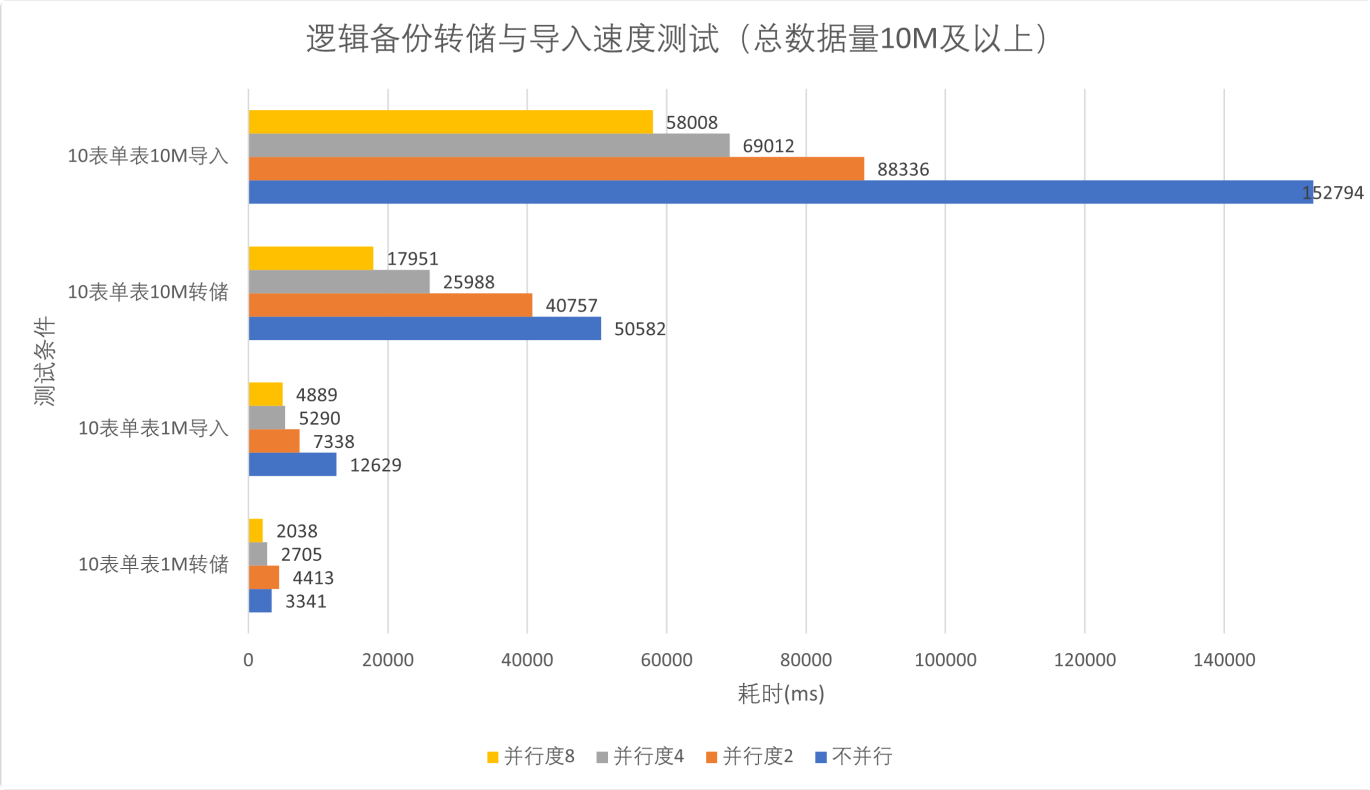
单表	1k	10k	100k	1M	10M
不并行转储	0.140s	0.207s	0.754s	12.629s	152.794s
并行转储并行度2	0.260s	0.314s	0.595s	7.338s	88.336s
并行转储并行度4	0.329s	0.307s	0.478s	5.290s	69.012s
并行转储并行度8	0.407s	0.417s	0.490s	4.889s	58.008s

数据库仅有1张表时数据导入速度测试

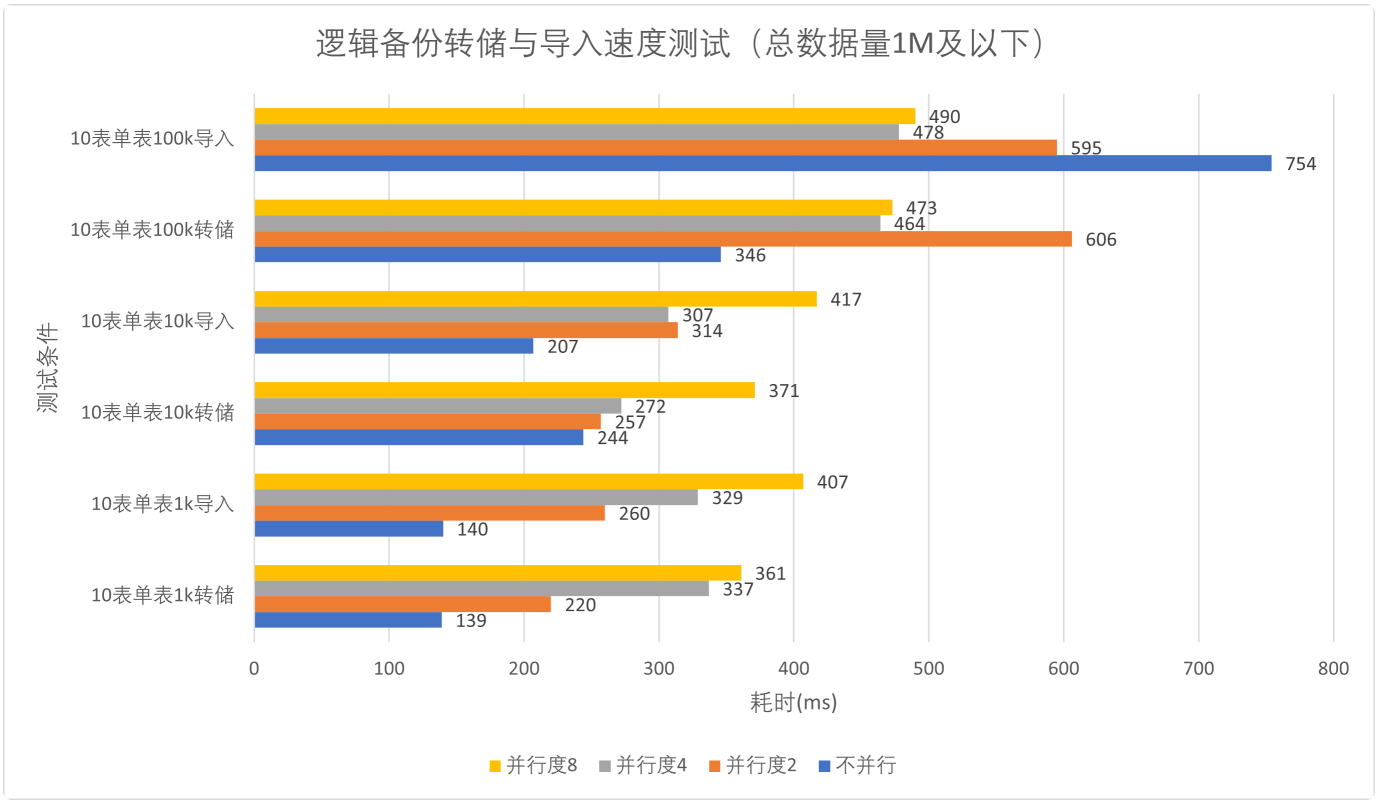
单表	10k	100k	1M	10M	100M
不并行转储	0.130s	0.191s	0.940s	9.447s	66.459s
并行转储并行度2	0.262s	0.457s	1.057s	11.022s	148.581s

并行转储并行度4	0.312s	0.431s	1.043s	10.165s	145.385s
并行转储并行度8	0.454s	0.432s	1.048s	11.199s	147.507s

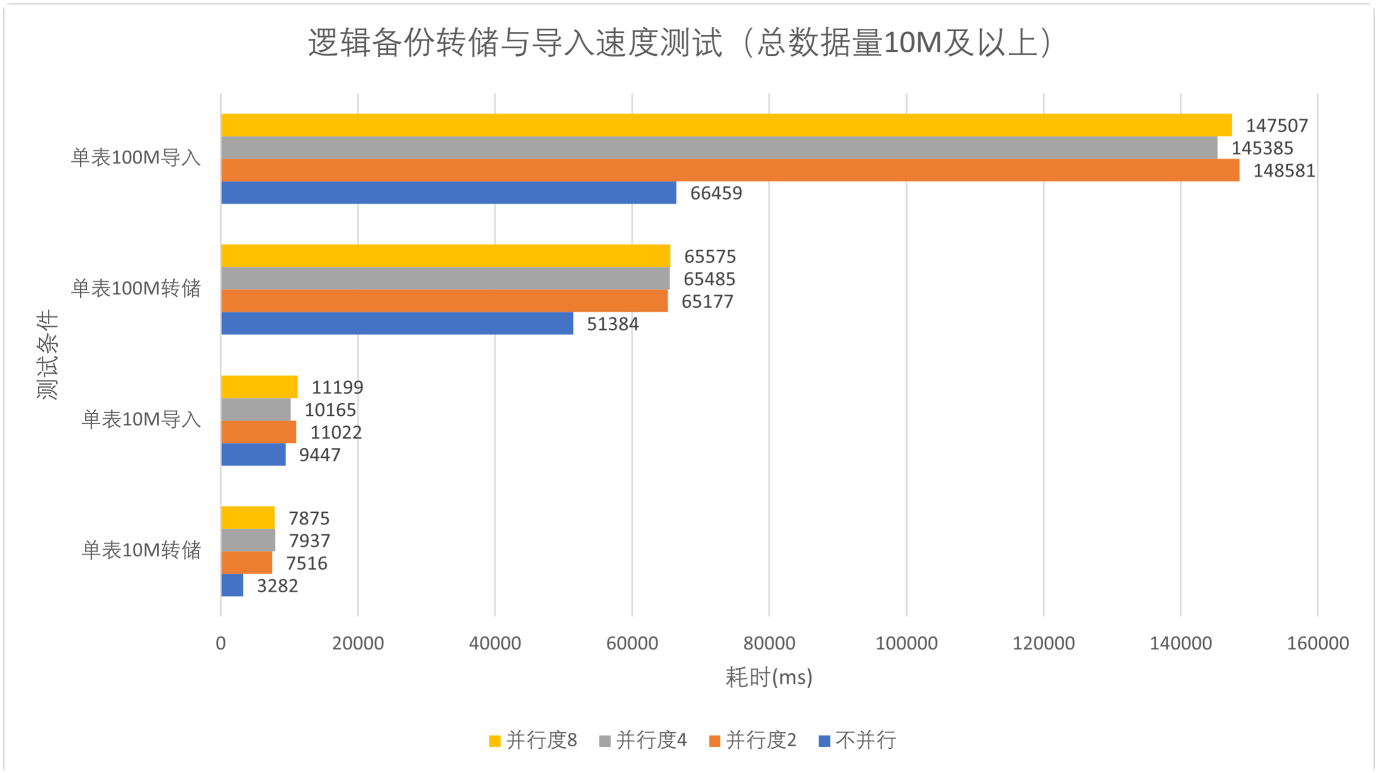
对上述测试结果进行数据处理绘制表格，结果如下



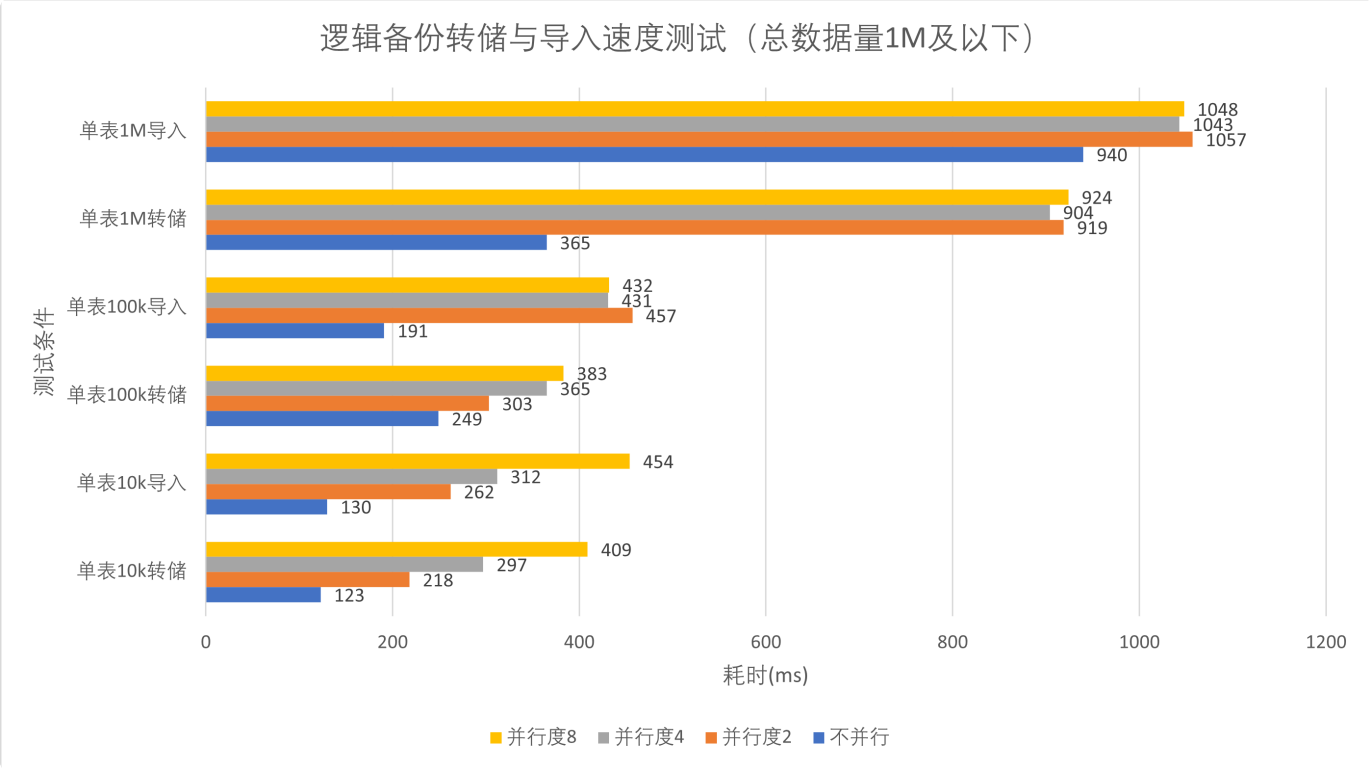
结果图1


逻辑备份转储与导入速度测试（总数据量1M及以下）
10表单表100k导入
10表单表100k转储
10表单表10k导入
10表单表10k转储
10表单表1k导入
10表单表1k转储
测试条件
490
478
595
754
473
464
606
346
417
307
314
207
371
272
257
244
407
329
260
140
361
337
220
139
0
100
200
300
400
500
600
700
800
耗时(ms)
并行度8
并行度4
并行度2
不并行

结果图2


逻辑备份转储与导入速度测试（总数据量10M及以上）
单表100M导入
单表100M转储
单表10M导入
单表10M转储
测试条件
147507
145385
148581
66459
65575
65485
65177
51384
11199
10165
11022
9447
7875
7937
7516
3282
0
20000
40000
60000
80000
100000
120000
140000
160000
耗时(ms)
并行度8
并行度4
并行度2
不并行

结果图3



结果图4

分析上述数据，我们可以由结果图1、3直观的看到，对于多表数据库，当数据量较大时，测试组中并行转储度越高，转储与恢复的速度越快。但对于单表数据库，开启并行转储会显著降低转储与恢复速度。这是由于PostgreSQL的并行转储与恢复操作的粒度是以表为单位的，对于多表数据库可以并行处理以加快转储恢复速度，但PostgreSQL不能对单张表进行并行转储或并行导入操作，反而会因为多线程开销降低转储与恢复速度。

同时我们由图2、4可以发现，当数据量较小时（总数据量1M条（24Mb）及以下），两种方式转出时间均较短（小于1.1s），对多表数据库的并行操作并未显著提高并行转储与导入效率，反而可能降低。

因此，对于**数据量较小，数据库表数较少的数据库，采用串行转储方式更加简单高效**。而对于**数据量较大的数据库，采用并行处理能够显著的提高转储速度**。与此同时，对于多表大型数据库，并行转储设置的并行参数的设置应参考主机配置与库表总数，**在主机配置许可的范围内尽可能增加并行度**，从而提高转储与恢复效率。

3. 逻辑备份操作对数据库运行影响测试

在线上环境中，数据库备份与转储的过程可能会对数据库当前运行的服务造成影响，从而可能提高数据备份操作的潜在成本，本研究对此设置实验进行探究。

实验huge.csv文件作为唯一表的测试数据，对单表数据库进行逻辑备份操作，同时在备份运行期间在其它连接中进行sql语句测试（每条sql语句作为独立事务运行），测试sql语句如下

测试语句 Shell

```
select count(*) from students where id between 1145 and 1926;
insert into students(studentid, name) values (100000001,'数据库');
update students set name='测试' where id=1;
delete from students where id=2;
truncate students;
```

在备份运行期间执行sql测试语句，执行结果如下

```
> select count(*) from students where id between 1145 and 1926
06] 1 row retrieved starting from 1 in 19 ms (execution: 4 ms, fetching: 15 ms)
> insert into students(studentid, name) values (1000000001,'数据库')
09] 1 row affected in 1 ms
> update students set name='测试' where id=1
11] 1 row affected in 1 ms
> delete from students where id=2
13] 1 row affected in 1 ms
> truncate students
10] completed in 54 s 433 ms
```

可以看到在备份期间所执行的增删改查语句并未受到显著影响，而对表本身操作的第五条sql语句执行耗时异常，这是由于备份期间备份线程对所需备份的表添加了访问锁（Access Shared Lock）[4]，该锁不会阻塞对表的DML语句操作与DQL语句查询，但是会阻塞对表定义的DDL语句操作，因此，在上述操作语句中，仅有truncate语句运行耗时过长，即该语句对表操作被逻辑备份所创建的表访问锁阻塞。

尝试对该表进行恢复导入，观察导入后的数据表

id	studentid	name
1	1	平保民
2	2	廉欣汝
3	3	陶昕怡
4	4	江司诚

可以看到update与delete操作并未生效

我们可以发现，在备份期间对数据库进行的DML操作并未生效。这是由于PostgreSQL采用了MVCC机制实现数据库的并发控制，逻辑备份指令运行时，会开启事务并创建运行指令时刻的数据库快照，该事务为可重复读隔离级别，备份事务将会读取历史快照数据进行备份，从而避免了对数据库加锁而阻塞DML操作。因此，在备份事务开启后进行的DML操作不会被备份，同时也不会影响逻辑备份事务的备份操作。

因此实验发现，**在逻辑备份运行期间，对数据库表的DDL操作会受到阻塞**，这是由于备份事务开启时对所有所需备份表均添加了表访问锁，在实际维护中，可以在**备份数据库期间添加-t参数指定所需备份的表**[4]。其次，PostgreSQL采用MVCC机制实现备份事务对表的可重复读，在备份期间避免了对DML语句

的运行进行阻塞，同时，在备份事务运行期间与运行结束后对数据库数据进行的操作不会被备份事务所备份。

PITR备份

为实现对事务模型的支持，PostgreSQL数据库系统采用预写日志（WAL）以实现对事务的回滚操作。连续归档和时间点恢复备份技术（PITR）便是基于时间点与WAL日志归档的备份恢复技术。在发生数据损坏时，PostgreSQL会依据基础备份与WAL日志对数据进行恢复，能够有效将数据内容恢复至故障发生前的最后有效数据，从而避免了备份点至故障发生点期间的数据丢失。实验对PITR技术的有效性进行了测试。

PITR备份有效性实验

实验采用huge.csv前1000行数据作为表数据进行模拟实验，实验过程如下

1. 首先，采用PITR数据备份技术需开启WAL日志归档，以长期存储WAL日志信息

在postgresql.conf中修改如下配置，随后重启服务器

配置文件示例 Shell

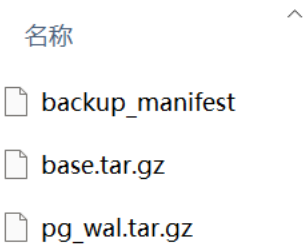
```
wal_level = archive
archive_mode = on
archive_command = 'copy "%p" "E:\\data\\%f"'
```

2. 执行如下指令为整个数据库集簇创建一个基础备份

备份指令 Shell

```
pg_basebackup -D E:\data\202310120100 -Ft -z -P -U postgres -Xs
```

可以在数据库中看到基础备份文件



3. 在基础备份创建完成后，执行如下测试语句，模拟对数据库数据的操作

测试语句 Shell

```
select count(*) from students where id between 1145 and 1926;
insert into students(id,studentid, name) values (1001,'1001','数据库');
update students set name='测试' where id=1;
delete from students where id=2;
```

4. 对数据库文件进行操作，模拟数据库文件的损坏

调用如下指令，查询数据库表文件的存储位置，并删除该存储文件

查找文件存储位置 Shell

```
select pg_relation_filepath('students');
```

运行select语句测试，可以看到数据库抛出异常

```
public> select * from students
2 17:58:13] [58P01] 错误: 无法打开文件 "base/26817/32936": No such file or directory
```

5. 开始采用连续归档备份对数据进行恢复

   a. 清空data路径下无用的数据目录文件与日志文件

   b. 将基础备份文件拷贝到数据目录下并解压，执行如下解压指令

```
tar -zxvf base.tar.gz
```

   c. 在postgresql.conf中添加恢复指令参数，添加如下配置

```
restore_command = 'copy "E:\\data\\%f" "%p"'
```

   d. 创建recovery.signal文件，可以在其中指定所需恢复到的时间点参数

   e. 启动服务器，此时服务器开始进行恢复工作

   f. 回到数据表中，我们检查students数据表中的数据信息

	id	studentid	name
1	1	1	测试
2	3	3	陶昕怡
3	4	4	江司诚
4	5	5	蔡方逍
5	6	6	岑苒溪
6	7	7	柳夫子
7	8	8	于涵越
8	9	9	俞安娜
9	10	10	顾易轩
10	11	11	卫建林
11	12	12	范益冉
12	13	13	韦涵越
13	14	14	卫美玲
14	15	15	李欣欣

	id	studentid	name
1	1001	1001	数据库
2	1000	1000	岑DBMS Perform…
3	999	999	褚苒溪
4	998	998	时溶溶
5	997	997	柏原神
6	996	996	姜国栋

可以发现所有的数据都被成功恢复，包括在备份点后进行的数据操作也被成功恢复。

根据上述实验，我们可以看到，采用PITR对数据库系统进行备份能够**有效地将数据恢复至数据库运行故障前的最后有效时刻，从而最大程度地保留了有效数据，有效保障了数据的一致性与持久性**。查阅PostgreSQL官方文档，我们同时发现，PITR备份运行时无需对数据库添加任何级别的锁，这是由于PostgreSQL系统在备份开始前先执行了一次检查点将数据刷入磁盘中，随后采用流复制技术从磁盘中复制数据，且在备份结束前阻止再次执行检查点刷入数据，从而在不阻塞数据库服务的情况下保证了备份数据的一致性。同时，pg_basebackup默认会将检查点所需I/O操作分散在一段时间上，从而避免集中I/O操作占用大量数据库性能资源影响数据库的正常服务。最后，除将数据恢复至崩溃前最后时刻外，**PITR备份还支持将数据回滚至指定的时间**，能够有效在开发过程中不慎写入脏数据后对数据进行恢复，有效保障了数据库的一致性[4]。

但是，PITR备份亦存在**数据备份与恢复操作较繁琐，存储文件体积较大，无法指定单个数据库或表进行备份的缺点**，同时，相较于逻辑备份，PITR备份所生成的数据间具有关联性，若备份文件损坏，则难以从中提取出有效数据。最后，PITR备份方式不具备跨版本与跨数据库能力，因此，使用逻辑备份或PITR备份的选择需参考现实开发与运营的需要。

块数据校验和

除数据的直接丢失或无法解析可能对数据一致性与持久化造成威胁外，数据库数据也可能会出现块损坏等数据异常，出现行损坏、信息丢失或重复，由于此类小部分数据的损坏不会直接造成数据库的运行异常，因此该故障往往十分隐蔽，难以被开发人员及时发现，因此对数据一致性造成了较大威胁。对于高价值低容差的数据存储场景，需采用某种手段对数据的正确性进行校验，PostgreSQL系统提供了块数据校验和的技术以实现此需求。

块数据和校验技术通过数据校验算法在对数据库进行修改操作时计算所存储数据块的校验和并保存，随后在每次读取该块时重新计算数据库校验和与已有校验和进行比较，以检验数据块是否损坏。实验对块数据校验和技术的有效性进行测试。

块数据校验和有效性实验

实验基于students表结构进行修改作为测试表，对块数据校验和的有效性进行测试，实验过程如下

1. 暂停数据库运行，开启块数据校验和，运行如下指令

```
pg_checksums -D "C:\Program Files\PostgreSQL\15\data" -e
```

2. 恢复数据库运行，创建students表并向其中插入测试数据

创建测试数据 SQL

```sql
create table students(
  id        int,
  studentid varchar(10),
  name      varchar
);
insert into students(id,studentid, name)  VALUES (1,'1','test');
```

3. 模拟数据库表文件发生数据损坏
    a. 运行pg_relation_filepath函数查询数据库表文件的存储位置
    b. 查看该页文件的内容，我们可以看到该数据表存储文件大小为8k，包含有一个数据块

```
00001eb0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001ec0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001ed0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001ee0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001ef0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f00  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f10  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f20  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f30  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f40  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f50  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f60  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f70  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f80  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001f90  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001fa0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001fb0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001fc0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00001fd0  00 00 00 00 00 00 00 00-13 42 03 00 00 00 00 00  .........B......
00001fe0  00 00 00 00 00 00 00 00-01 00 03 00 02 08 18 00  ................
00001ff0  01 00 00 00 05 31 0b 74-65 73 74 00 00 00 00 00  .....1.test.....
```

c. 复制该块数据并追加至该块末尾。在Windows系统下，实验选择该数据复制该数据表文件（41178_1），执行如下指令，实现追加复制数据块。

```sql
Get-Content 41178, 41178_1 -Encoding Byte | Set-Content 41178 -Encoding Byte
```

4. 执行select语句测试，发现PostgreSQL系统提示数据块页校验失败，表明数据库系统已检测非法数据

```sql
select * from students;
```

Data Output | Messages | Notifications

```
警告:  页校验失败，计算出的校验和为17462，但期望值是17461

ERROR:  关系 "base/41168/41178" 中的块 1 存在无效的页

错误:  关系 "base/41168/41178" 中的块 1 存在无效的页
SQL state: XX001
```

5. 停用数据块，执行pg_checksums检查数据页是否存在错误，执行如下代码

```
pg_checksums -D "(path)\data" --check
```

```
PS C:\Program Files\PostgreSQL\15> pg_checksums -D "C:\Program Files\PostgreSQL\15\data" --check
pg_checksums: error: checksum verification failed in file "C:\Program Files\PostgreSQL\15\data/base/41168/41178", block
1: calculated checksum 4436 but block contains 4435
Checksum operation completed
Files scanned:   4242
Blocks scanned:  28050
Bad checksums:  1
Data checksum version: 1
```

我们可以发现，pg_checksums检出了块数据异常的存在与异常存在位置。

6. 为验证数据块校验和的有效性，我们此时关闭checksums功能，进行查询测试

关闭数据块校验功能 Shell

```
pg_checksums -D "C:\Program Files\PostgreSQL\15\data" -d
```

再次执行select语句，我们发现查询结果包含了错误数据2，没有满足数据一致性需求。

```
select * from students;
```

Data Output | Messages | Notifications

	id integer	studentid character varying (10)	name character varying
1	1	1	test
2	1	1	test

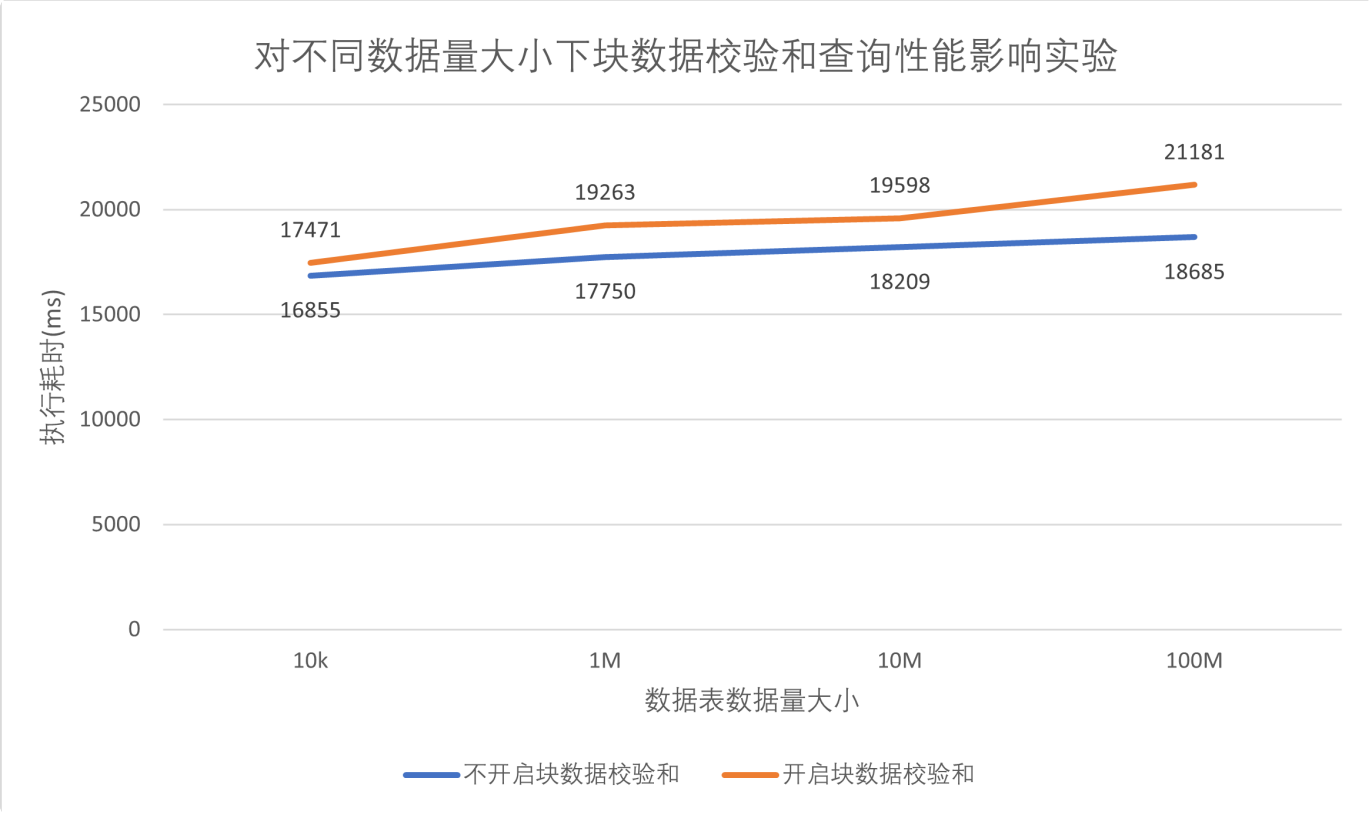
综合上述实验实验发现，开启块数据校验和能够及时有效地检出数据表中存在的数据损坏，从而在更高程度上保障了数据一致性与数据持久化。

块数据校验和性能影响实验

实验采用huge.csv前1万，前100万，前1000万与全表1亿数据，针对students表结构创建频繁查询文件select2.sql，对表进行10万次主键查询，比较开启块数据和校验与不开启块数据和校验对数据块频繁查询的性能影响，实验结果如下。

数据量	10k	1M	10M	100M
不开启块数据校验和	16.855s	17.750s	18.209s	18.685s
开启块数据校验和	17.471s	19.263s	19.598s	21.181s

对上述数据绘制表格进行分析，结果如下



由上述结果我们发现，开启数据校验和后数据查询操作所消耗的时间略高于未开启条件下的耗时，但不同数据量数据表执行相同频繁查询的时间差距基本于10%以内，表明块数据校验和对数据块查询性能存在轻微的影响

总结

综合上述实验我们发现，对于PostgreSQL所提供的三种备份方式，在实际使用中往往不提倡采用文件系统级的物理备份，而是采用更为实用有效的逻辑备份与PITR备份技术。逻辑备份能够有效地将数据备份至外部存储设备，其执行简单高效，且可以采用并行处理的方式提高多表大数据库的转储导入效率，但其需要执行定期备份，无法将数据精确的恢复至故障发生前的最后有效数据。而采用PITR备份技术能够有效地将数据恢复至故障发生前的最后有效时刻，同时也支持将数据回滚至指定的时间点，从而在更高的程度上保障了数据的一致性，但是，PITR备份亦存在操作较繁琐，存储文件体积较大，无法指定单个数据库或表进行备份，难以从中提取出有效数据，不具备跨版本与跨数据库能力等缺点。**因此，在实际的开发过程中，可以参考对数据完整性与一致性的实际需要，结合逻辑备份与PITR备份技术，在开PITR备份的同时，定期更新基础备份点与进行逻辑转储备份**。最后，**对于数据价值较高，数据一致性需求较高的场景，还可以通过开启块数据校验和功能，以从更高程度上保障数据的一致性**。

数据安全性探究

在保障数据库数据完整性、数据一致性的同时，数据安全性也是数据库管理中的重要部分。数据安全是指在数字信息的整个生命周期中保护数字信息免遭未经授权的访问并抵御损坏、失窃等风险[5]。相较于

采用文件系统进行简单管理，DBMS采取多种手段防止数据受损或泄露，维护数据的安全性，其中最基本的手段便是用户权限管理体系。

在用户权限管理体系下，开发人员可在开发阶段可针对需求设计用户权限，禁止用户对数据库的部分操作，从而阻止实际用户对数据进行非法访问或操作，保障数据的安全性。同时，开发团队也可为不同的开发人员分配不同的数据库用户并赋予不同权限，从而避免开发失误造成数据被破坏。

PostgreSQL系统采用角色的概念管理数据库访问，实现了完善的用户权限管理体系，本研究将对PostgreSQL系统的用户权限管理机制进行实验。

用户权限管理有效性实验

PostgreSQL数据块系统定义了一系列的对象权限，查阅PostgreSQL官方手册，我们可以总结出常用的数据权限关系并绘制成如下表格。在下表中，数据表行首表示所赋予的权限等级，列首表示可执行的数据库操作，使用√表示赋予后角色将拥有此项权限

	创建数据库	创建修改角色	select语句	insert语句	update语句	delete语句
superuser	√	√	√	√	√	√
createdb	√					
createrole		√				
select			√			
insert				√		
update					√	
delete						√

实验选取filmdb.sql生成测试用数据库对上述权限效果进行测试，测试过程如下

1. 创建一系列数据库用户并依照下表为其赋予权限

用户名	权限等级
super	superuser
creator	createdb、createrole
normal_user	none

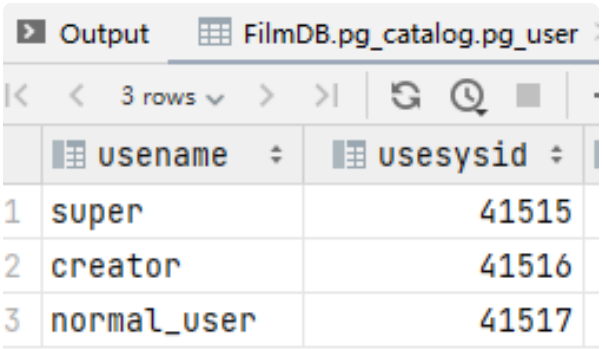
创建用户语句 Shell

```
create user super with superuser password 'pwdhere';
create user creator with createdb createrole password 'pwdhere';
create user normal_user with password 'pwdhere';
```

执行查询语句，检查所创建的用户

查询语句 Shell

```
select * from pg_user where usename in ('super','creator','normal_user');
```



	usename	usesysid
1	super	41515
2	creator	41516
3	normal_user	41517

可以看到用户已被成功创建。

2. 登陆用户并进行数据库操作尝试

   a. 登陆super用户，尝试执行数据库操作，操作语句如下

测试语句1 SQL

```
--创建与授权
create role test;
create database db;
grant select on movies to normal_user;
```

测试语句2 SQL

```
--撤权与删除
revoke select on movies from normal_user;
drop database db;
drop role test;
```

```
· create role test
.:38:07] completed in 3 ms
· create database db
.:38:07] completed in 116 ms
· grant select on movies to normal_user
.:38:07] completed in 1 ms
revoke select on movies from normal_user
38:35] completed in 1 ms
drop database db
38:35] completed in 56 ms
drop role test
38:35] completed in 1 ms
```

可以看到角色与数据库创建以及用户赋权的操作均能被正常执行

b. 登陆creator用户尝试执行 测试语句1，执行结果如下

```
ic> create role test
 21:43:15] completed in 4 ms
ic> create database db
 21:43:15] completed in 115 ms
ic> grant select on movies to normal_user
 21:43:15] [42501] 错误: 对表 movies 权限不够
```

可以看到，由于creator不存在对movies表的访问权限，因此creator无法对用户进行赋权操作

c. 登录到normal_user用户尝试执行 测试语句1，执行结果如下

```
ic> create role test
 21:56:05] [42501] 错误: 创建角色的权限不够
ic> create database db
 21:56:07] [42501] 错误: 创建数据库权限不够
ic> grant select on movies to normal_user
 21:56:10] [42501] 错误: 对表 movies 权限不够
```

可以看到，由于在初始创建时未对normal_user授予任何权限，因此norma_user无法执行测试语句1中的操作

3. 登陆数据库并进行数据操作测试

a. 登陆到normal_user用户，尝试执行 测试语句3，执行结果如下

测试语句3 SQL

```sql
select * from movies;
update movies set title='测试' where movieid=1;
delete from movies where movieid=1;
drop table movies;
```

```
· select * from movies
:02:29] [42501] 错误: 对表 movies 权限不够
· update movies set title='测试' where movieid=1
:02:31] [42501] 错误: 对表 movies 权限不够
· delete from movies where movieid=1
:02:33] [42501] 错误: 对表 movies 权限不够
· drop table movies
:03:00] [42501] 错误: 必须是表 movies 的属主
```

可以看到，由于normal_user对movies表并没有任何权限，因此上述sql语句操作均无法执行

b. 登陆到super用户，尝试对normal_user授予权限，授权代码如下

授权代码1 SQL

```sql
grant select on movies to normal_user;
```

再次登陆normal_user用户，尝试执行测试语句3，执行结果如下

```
ic> select * from movies
 22:06:25] 500 rows retrieved starting from 1 in 40 ms (execution: 5 ms, fetching: 35 ms)
ic> update movies set title='测试' where movieid=1
 22:06:40] [42501] 错误: 对表 movies 权限不够
ic> delete from movies where movieid=1
 22:06:42] [42501] 错误: 对表 movies 权限不够
ic> drop table movies
 22:06:44] [42501] 错误: 必须是表 movies 的属主
```

可以看到，当normal_user被授予对movies表的select权限后，查询操作得以正常进行，而其余操作因权限不足受阻

c. 登陆到super用户，尝试对normal_user进一步授予权限，授权代码如下

授权代码2 SQL

```sql
grant all on movies to normal_user;
```

随后登陆到normal_user，再次尝试执行测试语句3，执行结果如下

```
ic> select * from movies
 22:16:12] 500 rows retrieved starting from 1 in 52 ms (execution: 8 ms, fetching: 44 ms)
ic> update movies set title='测试' where movieid=1
 22:16:12] 1 row affected in 3 ms
ic> delete from movies where movieid=1
 22:16:12] [23503] 错误: 在 "movies" 上的更新或删除操作违反了在 "credits" 上的外键约束 "credits_movieid_fkey"
 22:16:12] 详细：键值对(movieid)=(1)仍然是从表"credits"引用的.
ic> drop table movies
 22:16:21] [42501] 错误: 必须是表 movies 的属主
```

在被授予了针对movies表数据的全部权限后，normal_user成功执行了查询与更新操作，但是尽管被授予删除权限，违反主键约束的非法删除操作依旧无法被执行。同时，all所赋予的权限并不包含DDL操作

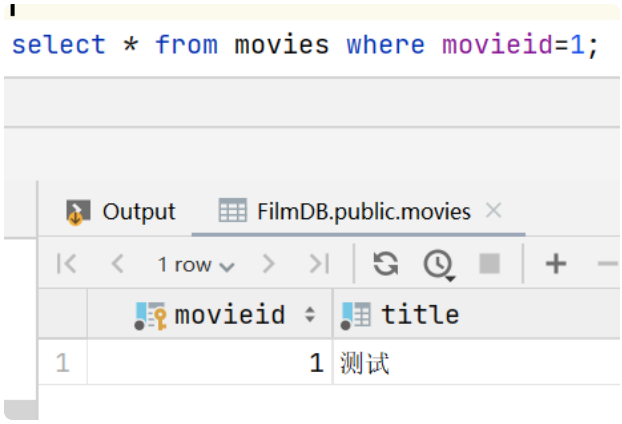
权限，查阅官方文档可以发信啊，PostgresSQL系统并不支持通过赋权的方式为用户单独授予对表的drop与alter权限，这是基于现实开发中常见的安全需求而设计的[4]。

4. 自收回权限设置

a. 登陆super用户，执行如下sql语句

测试语句4 SQL

```
select * from movies where movieid=1;
```



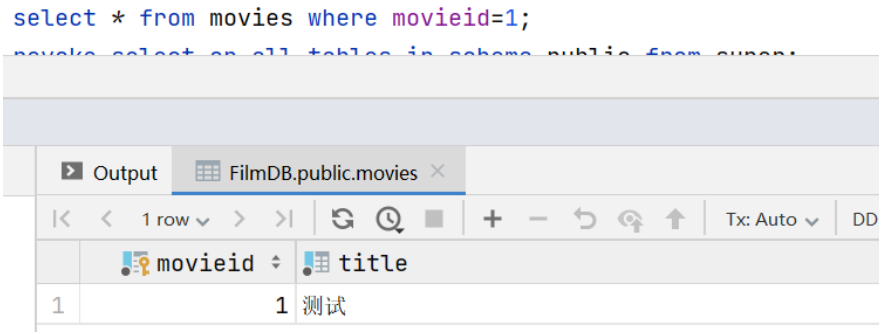
可以看到该语句可以被成功执行，这是由于super用户拥有超级用户权限，可以对任意表进行select操作

b. 执行如下指令，撤回super用户对所有表的select权限

SQL

```
revoke select on all tables in schema public from super;
```

c. 再次执行测试语句4，结果如下



我们发现，该super用户依旧能够进行查询操作，这是由于super用户为超级用户权限，对超级用户权限的撤回将不会生效，这样才能保证数据库权限管理体系的完整性。

通过上述实验我们可以看到，**PostgreSQL数据库系统设计了一套切实有效的权限管理机制**，通过对不同用户授予不同的数据操作权限，能够在一定程度上保障数据的安全性。

总结

通过上述对数据库基础数据操作、数据一致性、数据完整性、数据安全性四个方面的探究与实验，研究比较了数据库系统与传统文件读写间所存在的表现差异。研究发现，数据库系统相较于采用文件读写进行数据管理的方式具有较大的优势，其具体表现如下。

1. 对于基本数据操作，采用数据库系统能够方便有效地完成对数据的插入、查询、更新等基本操作，同时数据库系统支持采用多种技术优化数据的插入执行操作。同时对于实际开发运行中最常发生的数据查询操作，数据库系统采用了数据分页，缓存与索引等多种方式极大地优化了数据查询的效率，开发人员可以利用这些已经实现完备的技术有效提高程序的运行性能
2. 相较于手动编程进行文件读写操作，数据库系统能够有效地保证数据的一致性，借助事务模型与外键模型，数据库系统能够有效地保证数据的正确性，从而保证程序的正常运行。
3. 数据库系统有效地实现了对数据完整性的支持，能够过滤掉对于破坏表结构与数据约束的非法操作，同时，数据库系统提供了如逻辑备份、PITR备份恢复技术、块数据校验和等多种工具以保证数据的持久性，在一定程度上避免了数据的丢失、错误或者完整性受破坏。
4. 数据库系统实现了完善的权限管理机制，通过对不同用户授予特定的权限，数据库能够在一定程度上保障数据库的数据安全性。

综上，采用数据库系统能够更为有效的实现对数据的管理工作，使数据管理向着高可靠、高性能与规范化的方向发展。同时数据库系统同样处在在发展与更新之中，随着计算机技术的不断发展，数据库系统将进一步优化数据的存储过程与运行效率，增加数据库所支持的功能，提高数据库的易用性，从而为现代应用程序提供更为良好地数据管理服务。

参考文献

[1] Apache Spark as a Compiler: Joining a Billion Rows per Second on a Laptop

https://www.databricks.com/blog/2016/05/23/apache-spark-as-a-compiler-joining-a-billion-rows-per-second-on-a-laptop.html

[2] The Internals of PostgreSQL : Chapter 8 BufferManager

https://www.interdb.jp/pg/pgsql08.html

[3] What is Data Consistency? Definition, examples and best practice

https://www.decube.io/post/what-is-data-consistency-definition-examples-and-best-practice

[4] PostgreSQL官方文档

https://www.postgresql.org/docs

[5] What is data security –IBM

https://www.ibm.com/cn-zh/topics/data-security